Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по теме:

ВЕБ-ПРИЛОЖЕНИЕ ПЛАНИРОВАНИЯ ПИТАНИЯ С УЧЕТОМ ПРЕДПОЧТЕНИЙ ПОЛЬЗОВАТЕЛЕЙ

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Студент: _____ /Аксенов Кирилл Андреевич, 221–322/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Голубева Ирина Владимировна/
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Тема ВКР	Веб-приложение для планирования питания с учетом предпочтений пользователей
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Повышение эффективности управления тренировочным процессом за счёт автоматизации записи тренировок, визуализации прогресса и предоставления аналитики по физической активности пользователя.
Основные функции	1. Подбор и фильтрация рецептов по целям питания, ограничениям и пользовательским тегам. 2. Формирование персонального плана питания по дням и приемам пищи. 3. Расчет и отображение пищевой ценности рациона (калорийность, БЖУ). 4. Автоматическое формирование списка покупок на основе выбранного рецепта или плана питания. 5. Сохранение и управление избранными рецептами и планами питания. 6. Публикация пользовательского контента с последующей модерацией. 7. Уведомления о статусах модерации и ключевых событиях пользовательских сценариев. 8. Персонализированные рекомендации по рациону через встроенный AI-ассистент. 9. Административный и модераторский контур управления пользователями, жалобами и контентом.
Используемые технологии и платформы	База данных: PostgreSQL Серверная часть: Python, Django Клиентская часть: React, TypeScript, Effector

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Изучить предметную область и аналоги существующих решений 2. Выявить проблемы целевой аудитории, изучить аналоги существующих решений 3. Разработать требования к системе 4. Спроектировать архитектуру веб-приложения приложения 5. Спроектировать структуру базу данных 6. Разработать макеты пользовательского интерфейса 7. Разработать клиентскую и серверную часть приложения 8. Провести тестирование 9. Провести внедрение системы
Состав технической документации	1. Пояснительная записка.
Состав графической части	1. Общее число рисунков: 23 экз.; 2. Общее число таблиц: 7 экз.; 3. DFD-диаграмма: 2 экз.; 4. ER-диаграмма: 1 экз.; 5. BPMN-диаграмма: 1 экз.; 6. IDEF-диаграммы: 2 экз.; 7. Макеты интерфейса: 11 экз.

ПЛАН РАБОТЫ НАД ВКР

Этапы	Недели семестра																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Обзор предметной области																		
Определение целевой аудитории, её проблем, и изучение аналогов																		
Определение требований к разработке																		
Выбор архитектуры и набора используемых технологий для разработки клиентской и серверной частей.																		
Разработка серверной части веб-приложения																		
Разработка клиентской части приложения																		
Разработка демонстрационной версии																		
Тестирование																		
Установка на сервер																		

РУКОВОДИТЕЛЬ ОП:
«___»_____2026, _____/ Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:
«___»_____2026, _____/ Голубева Ирина Владимировна /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:
«___»_____2026, _____/ Аксенов Кирилл Андреевич, 221–322/
подпись *ФИО, группа*

АННОТАЦИЯ

Выпускная квалификационная работа выполнена на тему: «Веб-приложение для планирования питания с учетом предпочтений пользователей».

Назначение разработанного программного продукта состоит в автоматизации планирования рациона питания, формировании устойчивых здоровых пищевых привычек и повышении эффективности составления меню в повседневной практике пользователей.

Основные функции разработанного веб-приложения:

1. автоматический расчет пищевой ценности для каждого рецепта;
2. составление персональных планов питания на день, неделю или месяц (ручной и автоматический подбор);
3. формирование списка покупок на основе выбранных рецептов;
4. интеллектуальный ассистент для консультаций по питанию;
5. аналитика пищевых привычек и прогресса достижения пользовательских целей.

В работе используются следующие технологии: PostgreSQL (база данных), Python и Django (серверная часть), React и Effector (клиентская часть).

Работа включает введение, три главы, заключение, список использованных источников и приложения.

Во введении обоснована актуальность темы, сформулированы цель и задачи, определены объект и предмет исследования, а также практическая значимость результатов. В первой главе рассмотрена предметная область, проведен анализ целевой аудитории, ролевой модели и конкурентных решений, сформулированы требования к системе и обоснован выбор технологического стека. Во второй главе представлена реализация программного решения: проектирование структуры данных, разработка клиентской части, серверной части и прикладных модулей (планирование, расчет КБЖУ, модера-

ция, уведомления). В третьей главе описаны тестирование, контейнеризация и порядок развертывания системы.

Практический результат ВКР — работоспособное веб-приложение для планирования питания с поддержкой персонализации, автоматизации рутинных операций и ролевого управления контентом.

Практическая значимость разработанного решения состоит в снижении трудозатрат на составление рациона, повышении прозрачности контроля пищевой ценности и упрощении формирования списка покупок. Разработанная система может использоваться как самостоятельный цифровой инструмент для частных пользователей и как основа для дальнейшего развития сервисов в сфере здорового питания.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 8
1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ 11
1.1 Исследование предметной области 11
1.2 Определение целевой аудитории 11
1.3 Проектирование бизнес-процессов 13
1.4 Анализ аналогов и конкурентов 21
1.5 Определение требований к системе 22
1.6 Проектирование архитектуры системы 25
1.7 Выбор программных и технических средств разработки 36
1.8 Выводы по главе 38
2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ 39
2.1 Разработка пользовательского интерфейса приложения 39
2.2 Техническая реализация серверной части 49
2.3 Техническая реализация клиентской части 58
2.4 Вывод 64
3 ТЕСТИРОВАНИЕ, РАЗВЕРТЫВАНИЕ И ВНЕДРЕНИЕ СИСТЕМЫ . 65
3.1 Тестирование системы 65
3.2 Развертывание на сервере 66
3.3 Перспективы развития системы 68
3.4 Выводы по главе 69
ЗАКЛЮЧЕНИЕ 70
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 72

ВВЕДЕНИЕ

Вопросы, связанные с питанием и профилактикой неинфекционных заболеваний, являются устойчивым направлением исследований в сфере общественного здоровья. По данным Всемирной организации здравоохранения, факторы образа жизни, включая питание, рассматриваются как значимая составляющая профилактики и снижения риска хронических заболеваний [1, 2]. В национальных просветительских и профильных материалах по здоровому образу жизни данный тезис также подтверждается [3, 4]. В этой связи востребованы цифровые инструменты, позволяющие системно организовывать рацион и снижать долю рутинных операций при его планировании.

Для конечного пользователя задача планирования питания обычно включает несколько взаимосвязанных этапов: выбор подходящих рецептов, расчет пищевой ценности, составление меню на период, формирование списка покупок и последующий контроль выполнения плана. На практике эти действия нередко распределены между разными сервисами и инструментами, что усложняет процесс и увеличивает время выполнения базовых операций.

В условиях роста числа цифровых сервисов в сфере питания и фитнеса актуальной становится не только доступность отдельных функций, но и их интеграция в единую прикладную систему. С учетом этого проект ориентирован на создание веб-приложения, в котором ключевые пользовательские сценарии выполняются в рамках единого интерфейса и согласованной доменной модели.

Целью выпускной квалификационной работы является разработка веб-приложения для планирования питания с учетом предпочтений пользователей.

Для достижения поставленной цели необходимо решить следующие задачи:

1. исследовать предметную область;

2. определить функции продукта и технологии разработки;
3. проанализировать целевую аудиторию;
4. проанализировать аналоги и конкурентов;
5. спроектировать структуру базы данных;
6. спроектировать интерфейс;
7. разработать клиентскую часть;
8. разработать серверную часть;
9. разработать модули расчета КБЖУ, автоматического планирования и формирования списка покупок;
10. разместить веб-сервис на сервере.

Актуальность проекта обусловлена практической необходимостью автоматизации планирования рациона, которая позволяет сократить время на подбор рецептов и составление меню, уменьшить количество ручных ошибок при расчетах и повысить удобство соблюдения индивидуальных ограничений по питанию.

Объект исследования — процесс планирования питания, включающий подбор рецептов, учет пользовательских ограничений, расчет пищевой ценности, формирование планов питания и связанных с ними списков покупок.

Предмет исследования — методы и программные средства автоматизации планирования питания в формате веб-приложения.

Практическая значимость работы состоит в создании прикладного решения, позволяющего пользователю сократить время на планирование рациона, повысить прозрачность контроля КБЖУ и упростить формирование меню и списка покупок.

В результате выполнения работы разработано веб-приложение FoodPlanner, включающее клиентскую часть на React + Effector и серверную часть на Django + PostgreSQL. Реализованы базовые доменные контуры: управление рецептами и ингредиентами, конструктор планов питания, ролевой доступ, модерация пользовательского контента, уведомления, избранное и формирование списков покупок.

Соответствие цели и полученного результата подтверждается тем, что реализованная система покрывает ключевые пользовательские сценарии, определенные на этапе анализа предметной области, и обеспечивает основу для дальнейшего развития функционала персональных рекомендаций и аналитики пищевых привычек.

1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ

1.1 Исследование предметной области

На начальном этапе разработки рассмотрен практический сценарий пользователя, который стремится планировать рацион системно: подбирать рецепты, учитывать ограничения по питанию, формировать меню на период и получать список покупок. Анализ существующих пользовательских сценариев показывает, что эти действия часто распределены между несколькими сервисами: кулинарными порталами, приложениями учета калорий и отдельными инструментами ведения заметок.

При таком подходе возникают повторяющиеся операционные трудности: дублирование данных, ручной перенос информации между сервисами, отсутствие единой модели пользовательских предпочтений и ограничений, а также разрыв между этапом выбора блюда и этапом практического исполнения плана (покупка продуктов). В результате возрастает время на планирование и снижается управляемость процесса в долгосрочном периоде.

Предварительный обзор существующих решений показывает, что на рынке присутствуют как кулинарные платформы, так и специализированные сервисы учета питания, однако пользовательский сценарий часто остается разорванным между несколькими инструментами [5, 6, 7]. Детальный сравнительный анализ конкурентов и аналогов приведен в разделе 1.4.

1.2 Определение целевой аудитории

Целевая аудитория разрабатываемого веб-приложения включает группы пользователей, объединенные общей потребностью в системном управлении питанием, но различающиеся по сценариям использования.

Основная пользовательская аудитория

К основной аудитории относятся пользователи, планирующие рацион в соответствии с персональными целями:

1. снижение массы тела, поддержание формы или набор массы;

2. учет персональных ограничений в питании;
3. ускорение ежедневного планирования меню;
4. получение рекомендаций по выбору блюд и структуры рациона.

Ключевые требования этой группы:

1. персонализация рекомендаций и фильтрации;
2. визуально понятное планирование по дням и приемам пищи;
3. контроль КБЖУ и состава блюд [8];
4. формирование практического списка покупок на основе выбранного меню.

Авторы пользовательского контента

В данную группу входят пользователи, которые создают рецепты, публикуют планы питания и оставляют отзывы.

Для этой категории критичны:

1. удобные формы публикации и редактирования материалов;
2. прозрачность статусов модерации контента;
3. получение обратной связи от других пользователей.

Ролевые группы в системе

Ролевые группы описывают не целевую аудиторию, а модель разграничения прав доступа в программной системе.

Пользователь

Базовая роль, обеспечивающая работу с рецептами, планами питания, отзывами, избранным и списками покупок в рамках собственных данных.

Модератор

Роль контроля качества контента: проверка рецептов, отзывов и жалоб, принятие решений по модерации, корректировка публикаций в пределах регламента платформы.

Администратор

Роль расширенного системного управления: управление учетными записями и ролями, администрирование справочников, контроль устойчивости и безопасности функционирования платформы.

1.3 Проектирование бизнес-процессов

Бизнес-процесс представляет собой совокупность связанных активностей, преобразующих входные данные в результат, имеющий ценность для пользователя [9]. Для формализации процессов в рамках проекта используется подход структурного анализа и проектирования SADT (Structured Analysis and Design Technique), в составе которого применяется семейство нотаций IDEF [10, 11, 12].

Краткая характеристика основных нотаций IDEF приведена в таблице 1.

Таблица 1 – Краткая характеристика нотаций IDEF

Нотация	Назначение нотации	Основные отражаемые объекты
IDEF0	Создание функциональной модели	Функции, входы/выходы, управляющие воздействия и механизмы выполнения процессов.
IDEF1	Построение информационной модели	Информационные потоки и структура данных предметной области.
IDEF1X	Построение реляционных структур	Сущности, атрибуты и связи между сущностями в терминах реляционной модели.
IDEF3	Документирование сценариев выполнения процессов	Последовательность операций и варианты прохождения процесса.
IDEF5	Онтологическое описание сложных систем	Классы объектов, отношения, ограничения и изменения состояния.

Для проектирования ключевого процесса «Составление плана питания пользователем» в проекте использована нотация IDEF0, а для описания межролевого взаимодействия — BPMN 2.0 [10, 13, 14].

Нотация BPMN 2.0 поддерживает несколько типов представлений: оркестровка (один пул с внутренним потоком управления), диаграмма взаимодействия (несколько пулов и обмен сообщениями), хореография и схема диалога [13, 14]. В рамках FoodPlanner практическую ценность для описания

пользовательских и системных сценариев имеют оркестровка и диаграмма взаимодействия.

Диаграмма вариантов использования на рисунке 1 показывает распределение сценариев между ролями системы.

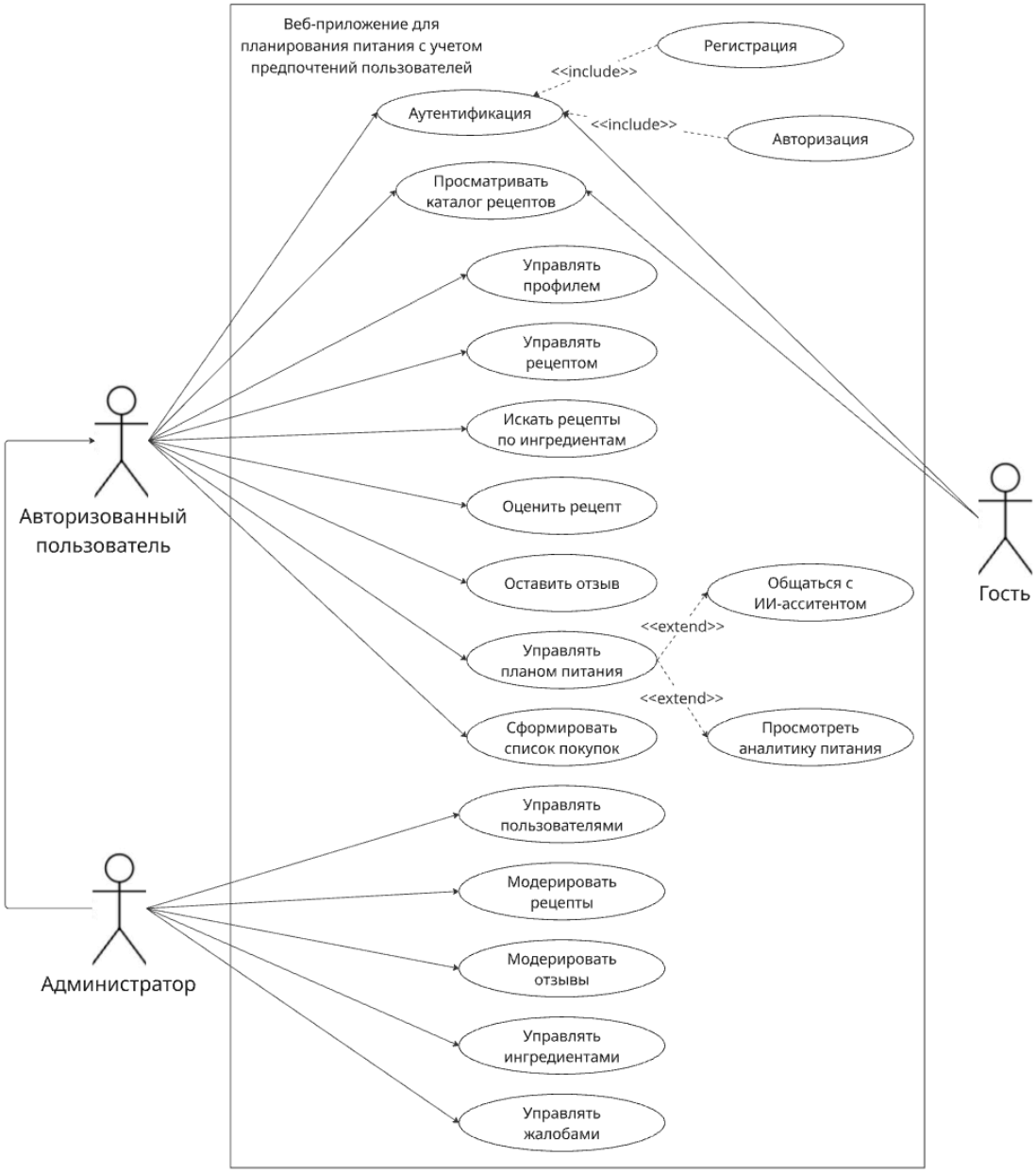

Рисунок 1 – UML-диаграмма пользователей

Неавторизованный пользователь (гость) имеет доступ к просмотру каталога рецептов и регистрации в системе. После прохождения регистрации пользователь получает возможность авторизации и перехода к персональным сценариям.

Авторизованный пользователь, помимо просмотра каталога, может редактировать профиль, указывать предпочтения, создавать и редактировать собственные рецепты, собирать планы питания, формировать список покупок, оставлять оценки и отзывы, а также использовать ИИ-ассистента и просматривать персональную аналитику.

Администратор обеспечивает управленческие и контрольные функции: управление пользователями и ролями, администрирование справочников, контроль жалоб и контроль корректности модерационных процессов.

Процесс взаимодействия пользователя с планом питания представлен BPMN-диаграммой (рисунок 2).

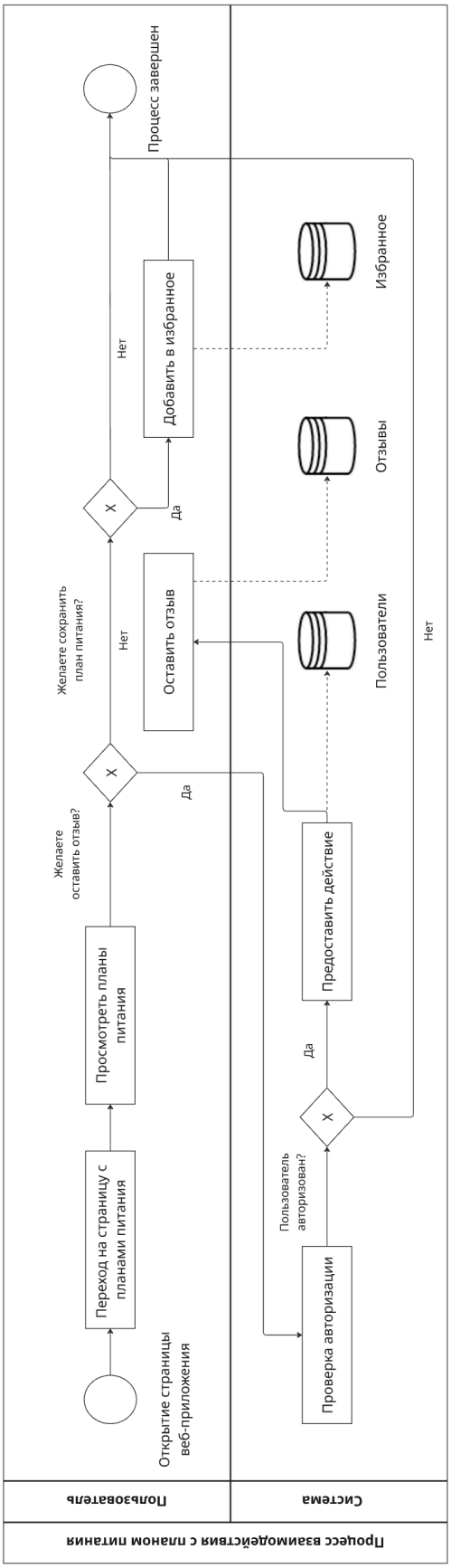

Рисунок 2 – BPMN-диаграмма процесса взаимодействия пользователя с планом питания

Алгоритм составления плана питания пользователем приведен на рисунке 3.

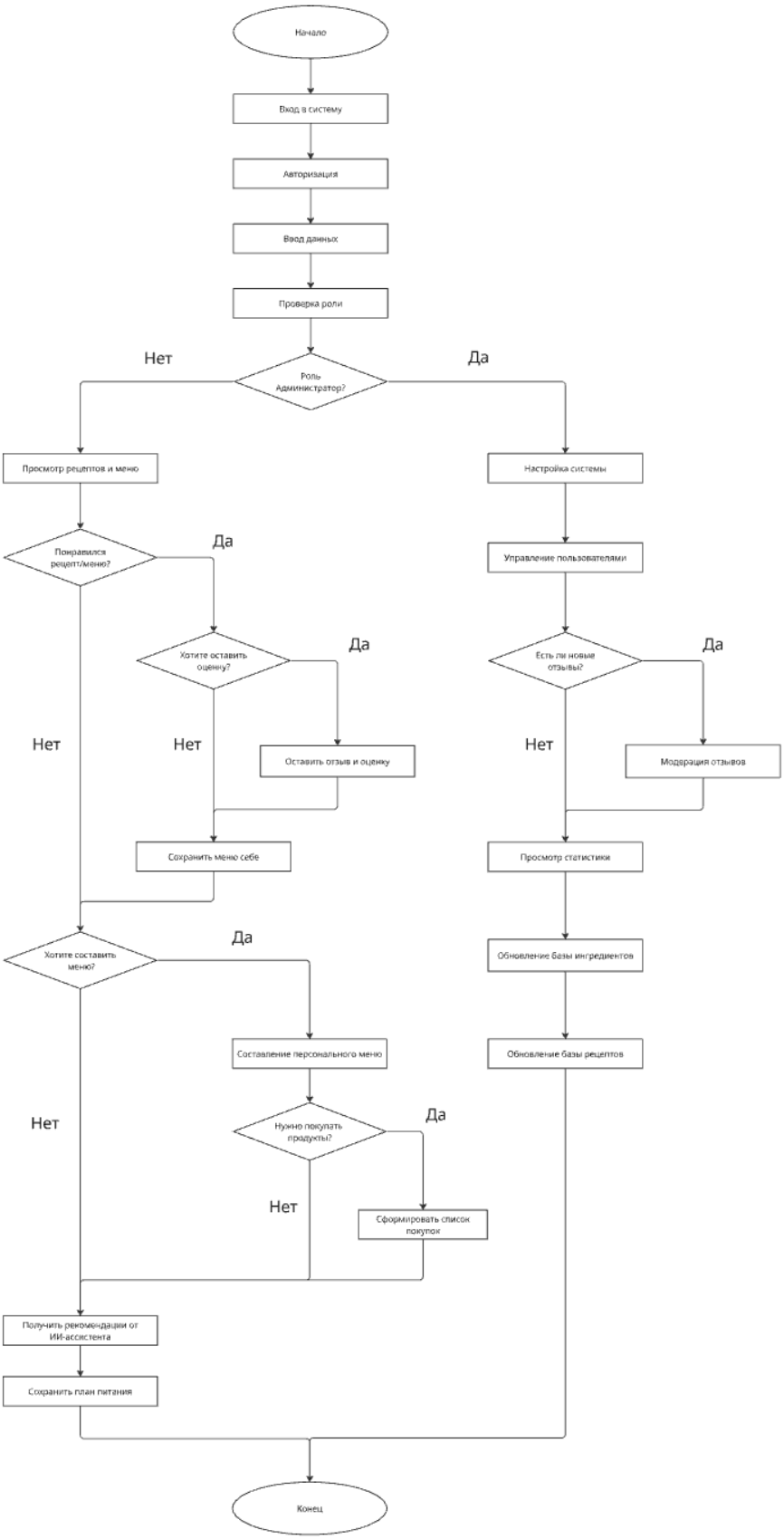

Рисунок 3 – Процесс создания плана питания

Взаимодействие пользователя с системой в рамках IDEF0

Пользователь задает исходные условия: доступные продукты, цели питания, ограничения и предпочтения. Эти данные поступают на вход функциональной модели. Далее система последовательно выполняет анализ входных данных, подбор рецептов, формирование меню и генерацию списка покупок с расчетом пищевой ценности.

Контекстная диаграмма IDEF0 (уровень A-0)

На контекстном уровне система представлена единой функцией «Планирование питания с учетом предпочтений пользователя». Входами являются данные о продуктах и целях питания; управлениями — ограничения, предпочтения и нормативные параметры; механизмами — веб-интерфейс, сервер приложений и база данных; выходами — сформированное меню, список покупок и расчет пищевой ценности.

Декомпозиция IDEF0 (уровень A0)

Декомпозиция верхнего уровня включает четыре функциональных блока:

1. анализ имеющихся продуктов;
2. поиск подходящих рецептов;
3. выбор оптимального меню на период;
4. формирование списка недостающих покупок.

Диаграммы IDEF0 и IDEF1 приведены на рисунке 4 и рисунке 5.

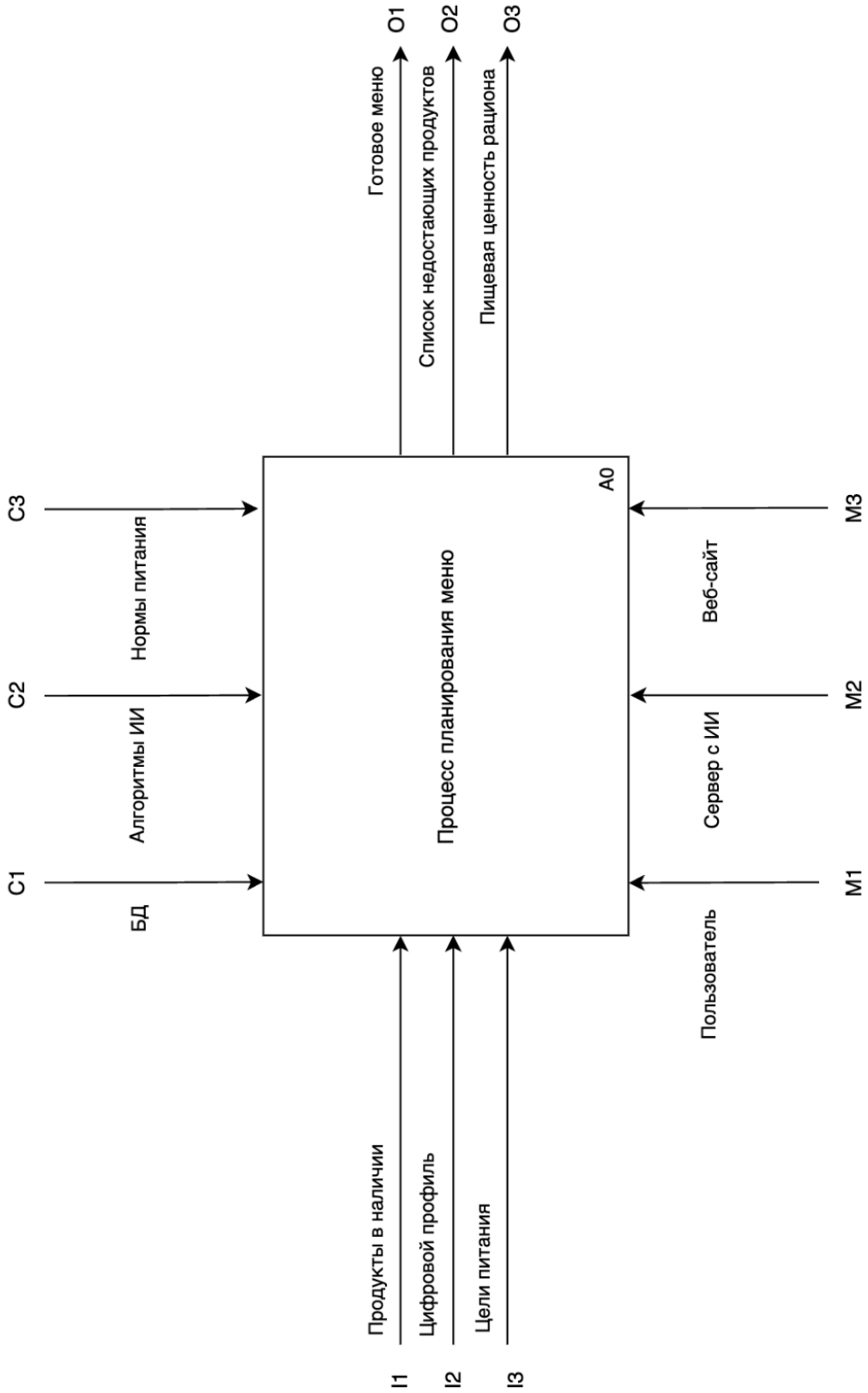

Рисунок 4 – IDEF0-диаграмма бизнес-процесса

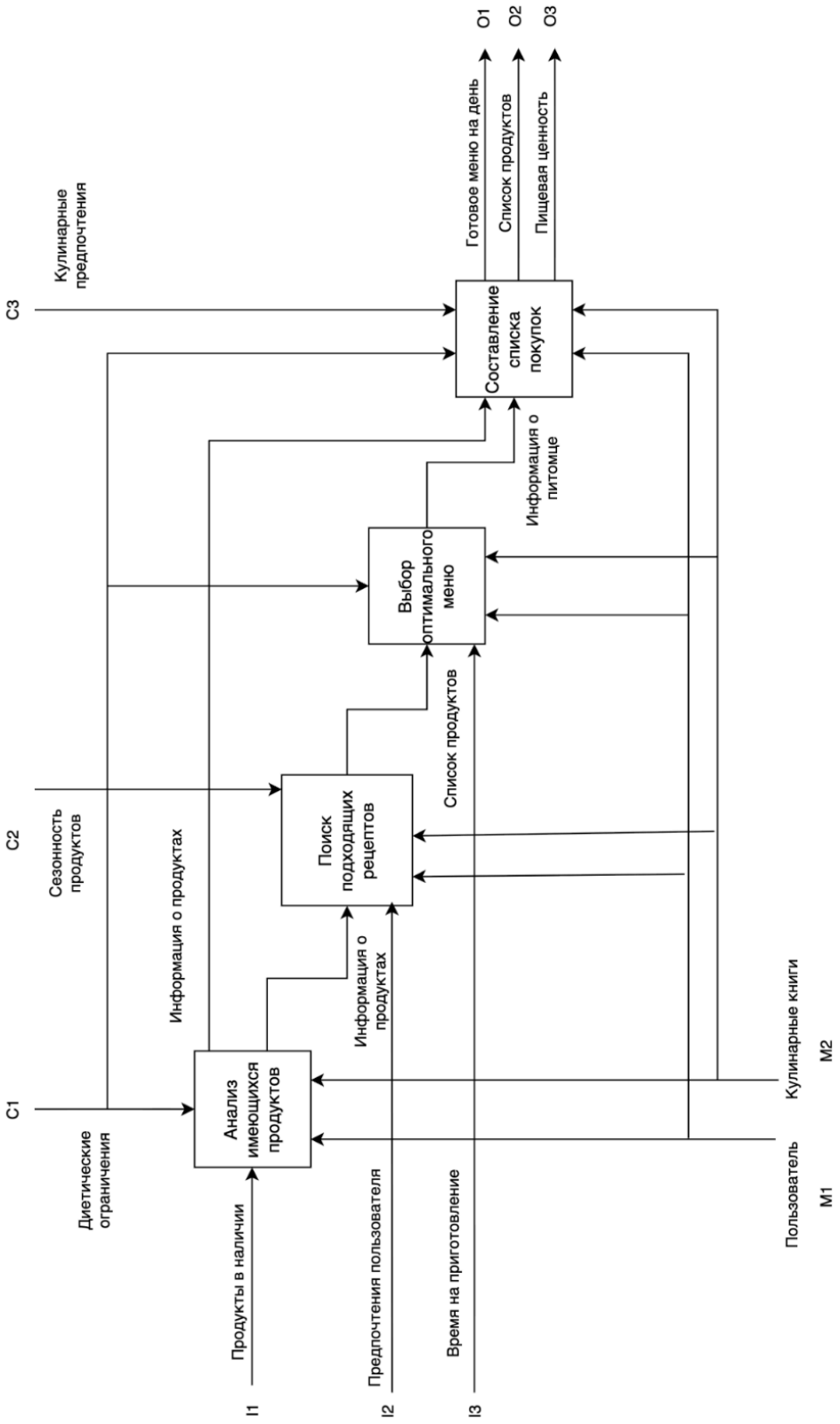

Рисунок 5 – IDEF1-диаграмма структуры данных

1.4 Анализ аналогов и конкурентов

Анализ аналогов и конкурентов выполнен для оценки уровня покрытия пользовательских потребностей, выявленных в предыдущих разделах: целостное планирование рациона, персонализация, учет ограничений, формирование списка покупок и поддержка модерации пользовательского контента. Дополнительно учтены обзорные публикации по приложениям и дневникам питания [5, 6, 7].

В качестве конкурентов рассмотрены кулинарные и nutrition-сервисы с широкой пользовательской базой: «Поварёнок», «Еда.ру», MyFitnessPal [15, 16, 17]. В качестве функциональных аналогов рассмотрены сервисы с выраженными отдельными возможностями в части подбора блюд и базового планирования: SuperCook и Ovkuse [18, 19].

Таблица 2 – Сравнение конкурентов и аналогов

Сервис	Сильные стороны	Ограничения для целевого сценария
Поварёнок	Большой каталог пользовательских рецептов и развитое сообщество	Отсутствует целостный контур персонального планирования и автоматического формирования списка покупок.
Еда.ру	Качественный редакционный контент и удобная навигация по рецептам	Фокус на контенте, а не на полном цикле планирования питания и контроля прогресса.
MyFitnessPal	Сильный учет калорий и макронутриентов, дневник питания	Нет полноценного сценария планирования меню на период и связанного списка покупок.
SuperCook	Эффективный поиск блюд по имеющимся ингредиентам	Узкая специализация, отсутствуют функции модерации и комплексного ведения рациона.
Ovkuse	Базовые функции подбора рецептов и формирования покупок	Ограниченный горизонт планирования и недостаточная персонализация под долгосрочные цели.

Результаты анализа показывают, что существующие решения закрывают отдельные этапы пользовательского пути, но не обеспечивают единый контур «от выбора рецептов до исполнения плана и контроля результата». Это подтверждает выбор проектного направления: разработка собственного веб-приложения с интеграцией планирования, учета КБЖУ, списков покупок, ролевой модерации и персонализированных рекомендаций в рамках одной системы.

1.5 Определение требований к системе

В данном разделе требования к системе представлены в порядке бизнес-приоритета: бизнес-требования, бизнес-правила и функциональные требования.

Бизнес-требования

Бизнес-требования сформулированы в измеримом виде (базовое значение → целевое значение за 6 месяцев после запуска):

1. удержание пользователей D30: 18% → 30%;
2. конверсия из просмотра каталога в создание плана питания: 12% → 25%;
3. доля пользователей, формирующих список покупок после создания плана: 20% → 40%;
4. среднее время модерации пользовательского контента: 72 ч → 24 ч;
5. доля жалоб, обработанных в пределах SLA 48 ч: 60% → 90%;
6. доля контента с прозрачным отображением статуса модерации: 70% → 100%.

Бизнес-правила

Бизнес-правила задают обязательные ограничения на поведение системы:

1. ролевая модель обязательна: действия модерации доступны только ролям moderator и admin;

2. жизненный цикл пользовательского контента фиксирован: `draft` → `pending` → `approved/rejected`;

3. пользователь может редактировать и удалять только собственный контент; администратор имеет расширенные права;

4. при заполненном профиле предпочтений система обязана учитывать его в персонализированной выдаче;

5. список покупок сохраняется с привязкой к источнику (рецепт или план питания) и доступен для повторного использования;

6. изменения статусов модерации и ответы по жалобам сопровождаются созданием уведомлений;

7. ключевые действия в процессах модерации и администрирования подлежат журналированию в аудите.

Функциональные требования

Для реализации бизнес-целей выделены ключевые подсистемы, представленные в таблице 3.

Таблица 3 – Ключевые подсистемы FoodPlanner для реализации функциональных требований

Название подсистемы	Описание
Система управления аккаунтами и доступом	Регистрация, авторизация, управление профилем и ролевым доступом.
Система каталога и персонализации	Просмотр рецептов и планов, поиск, фильтрация и персонализированная выдача.
Система конструктора плана питания	Формирование меню по дням и приемам пищи, настройка параметров, отправка на модерацию.
Система модерации и обратной связи	Модерация рецептов, планов и отзывов, обработка жалоб, фиксация решений.
Система избранного и списков покупок	Сохранение рецептов/планов в избранное, формирование и хранение списков покупок.
Система уведомлений и ИИ-ассистента	Уведомления о событиях платформы и диалоговый интерфейс рекомендаций.

На основании указанных подсистем сформулированы следующие функциональные требования:

1. пользователь может зарегистрироваться в системе и выполнить вход по учетным данным;

2. пользователь может просматривать и редактировать профиль (имя, фото, цели, особенности питания, теги);

3. система поддерживает ролевую модель `user/moderator/admin` с разграничением прав;

4. пользователь может просматривать каталоги рецептов и планов питания с поиском, фильтрацией и пагинацией;

5. пользователь может создавать план питания в конструкторе и отправлять его на модерацию;

6. модератор/администратор может принимать решения по модерации контента;

7. пользователь может оставлять оценки и отзывы для рецептов и планов питания;

8. пользователь может формировать и сохранять списки покупок с привязкой к рецепту или плану;

9. система отображает уведомления о результатах операций и статусах модерации;

10. пользователь может получать рекомендации через встроенный ИИ-ассистент.

Нефункциональные требования

Ключевые нефункциональные требования:

1. доступность сервиса в production: не ниже 99.5% в месяц;

2. медианное время ответа API для типовых запросов каталога: не более 500 мс;

3. время загрузки ключевых страниц интерфейса при стандартном канале: не более 2 с;

4. масштабируемость базы данных и API при росте объема данных не менее чем в 10 раз;

5. безопасность: CSRF, ограничение CORS, хэширование паролей, HTTPS, отсутствие ключей внешних API на клиенте [20, 21, 22, 23, 24];

6. сопровождаемость: модульная структура frontend/backend и документированные контракты API.

1.6 Проектирование архитектуры системы

Общая структура данных в приложении

Общая структура данных и потоков в FoodPlanner может быть формализована через DFD-представление. Контекстная диаграмма отражает взаимодействие системы с ключевыми внешними сущностями:

1. пользователь платформы (формирование рациона, работа с рецептами и планами);
2. модератор и администратор (контроль качества контента и управление системой);
3. клиентское веб-приложение (SPA), выступающее основным каналом пользовательского взаимодействия;
4. внешние сервисы (например, API ИИ-ассистента).

Контекстная DFD-диаграмма представлена на рисунке 6.

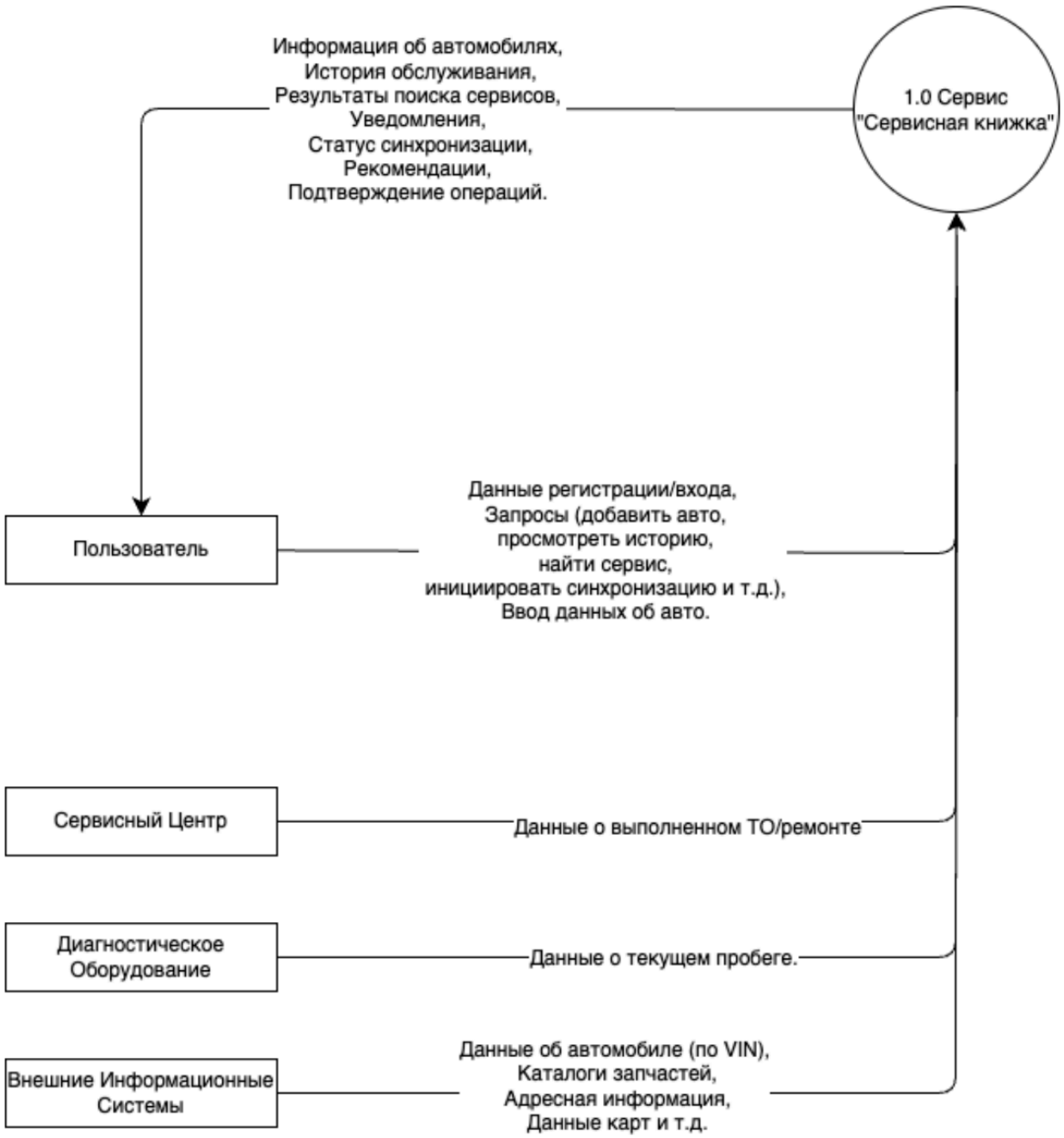

Рисунок 6 – Контекстная DFD-диаграмма FoodPlanner

При детализации DFD-уровня выделяются основные процессы и хранилища данных.

Процессы:

1. управление пользователями и аутентификацией;
2. управление рецептами, тегами, категориями и ингредиентами;
3. управление планами питания и генерацией списков покупок;
4. модерация пользовательского контента и обработка жалоб;

5. управление отзывами, избранным и уведомлениями;
6. взаимодействие с внешним API ИИ-ассистента.

Хранилища данных:

1. DS1: учетные записи, роли и настройки профилей пользователей;
2. DS2: рецепты, планы питания, структура рационов и shopping list;
3. DS3: отзывы, жалобы, модерационные журналы и уведомления.

Детализированная DFD-диаграмма представлена на рисунке 7.

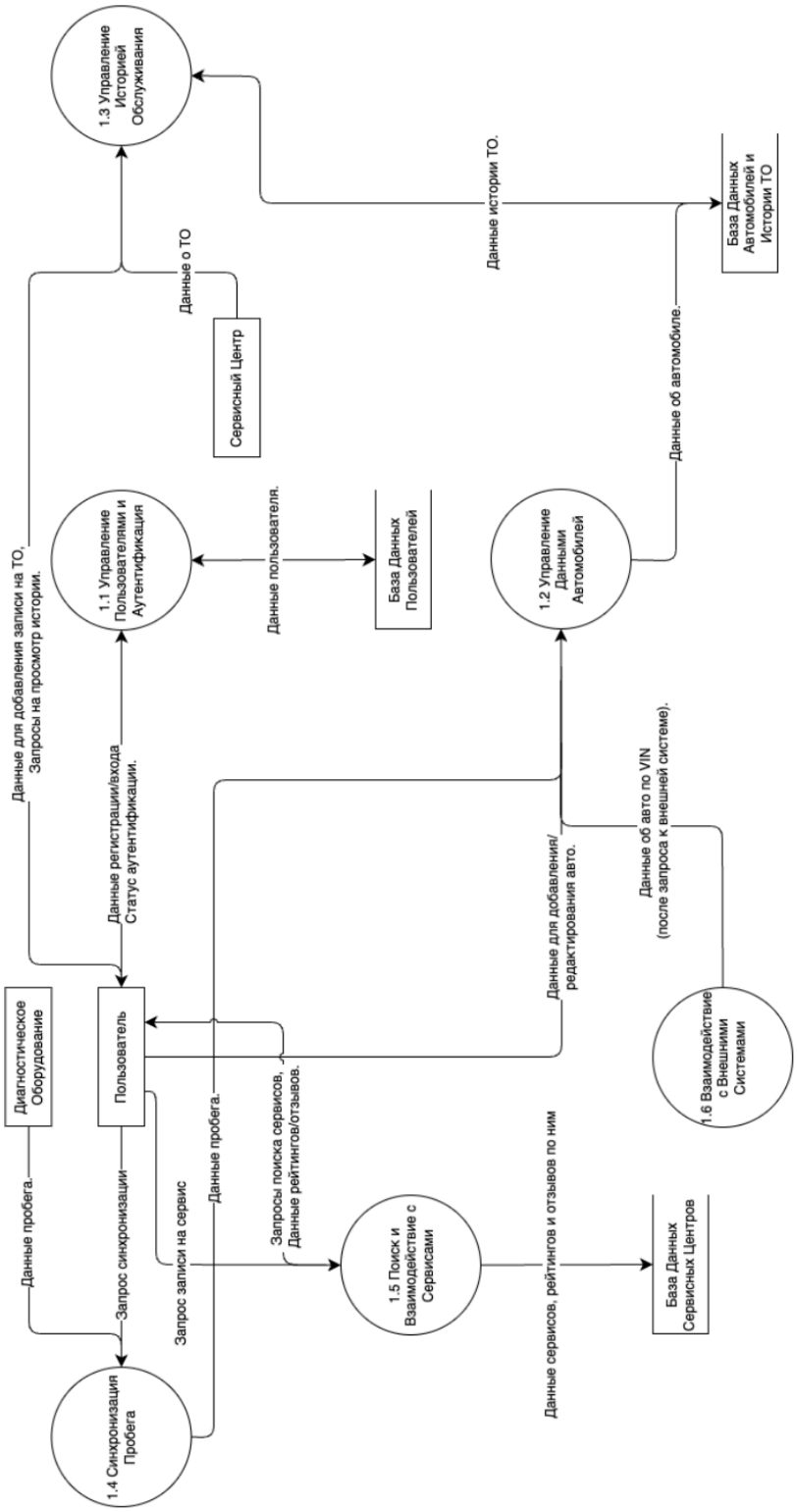

Рисунок 7 – DFD-диаграмма процессов и хранилищ данных FoodPlanner

Такая организация данных и потоков обеспечивает целостность информации и упрощает масштабирование системы при росте количества пользователей и контента.

Выбор архитектуры веб-сервиса

Для FoodPlanner выбрана клиент-серверная архитектура: клиентская часть (React + TypeScript) и серверная часть (Django + DRF) реализованы как независимые модули с единым API-контрактом. Альтернативным вариантом могла быть микросервисная архитектура, однако для текущего масштаба проекта и размера команды она избыточна по операционной сложности.

Схематичное представление клиент-серверной архитектуры и микросервисного подхода показано на рисунках 8 и 9.

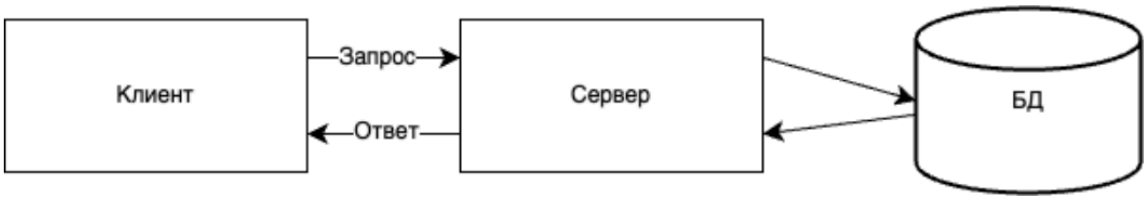

Рисунок 8 – Клиент-серверная архитектура FoodPlanner

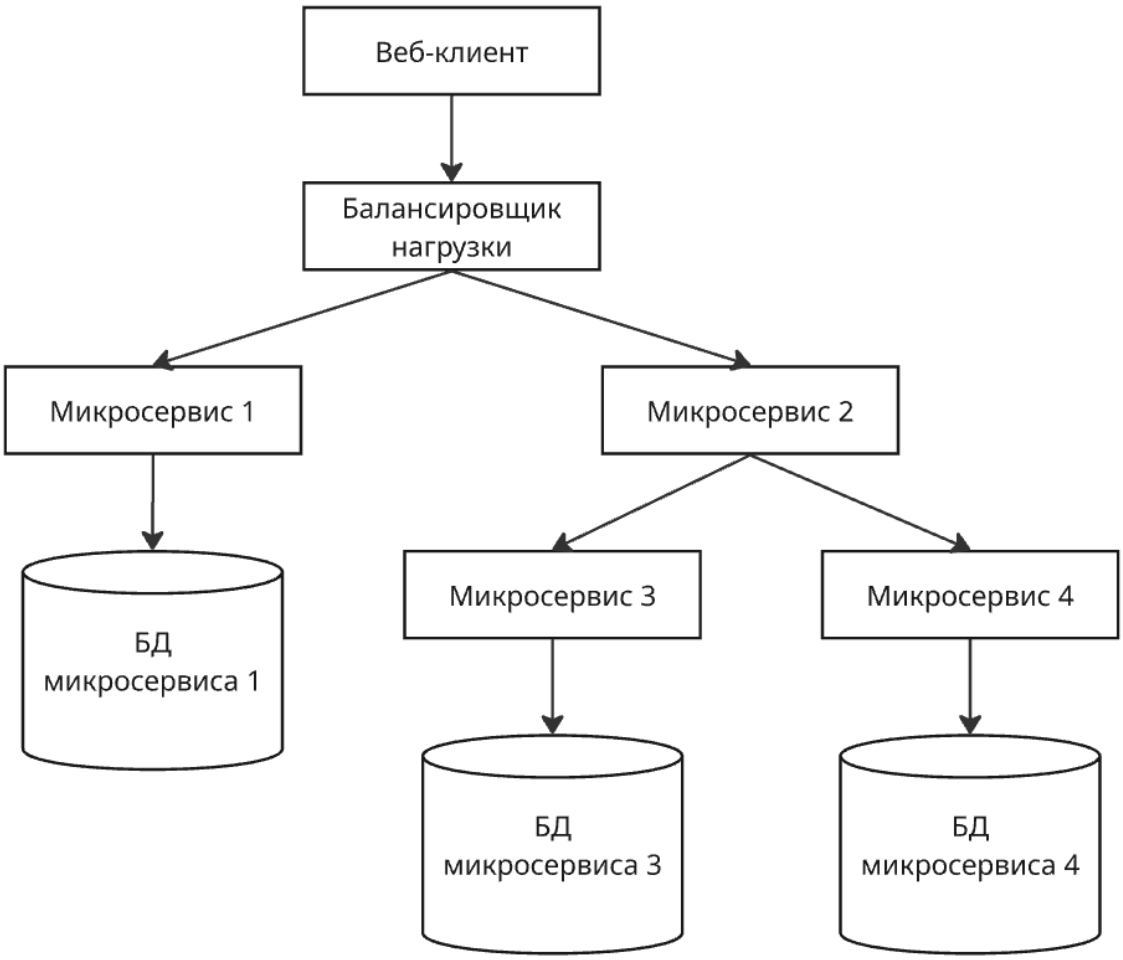

Рисунок 9 – Микросервисная архитектура (альтернативный подход)

Сравнительный анализ клиент-серверной и микросервисной архитектур приведен в таблице 4.

Таблица 4 – Сравнение клиент-серверной и микросервисной архитектур

Критерий	Клиент-серверная архитектура	Микросервисная архитектура
Принцип	Единый централизованный backend для обслуживания клиентов	Набор автономных сервисов, каждый отвечает за свою предметную область
Компоненты	SPA-клиент, API-сервер, единая БД	Набор сервисов, API-шлюз, межсервисное взаимодействие, распределенные БД
Масштабируемость	Масштабирование преимущественно на уровне всего backend	Независимое масштабирование отдельных сервисов
Развертывание	Единый релизный контур и более простой процесс деплоя	Независимые релизы, более сложная инфраструктура CI/CD
Сложность разработки	Ниже, проще старт и сопровождение малой командой	Выше из-за распределенности и усложнения инфраструктуры
Надежность при сбоях	Критические сбои затрагивают сервис целиком	Сбой одного сервиса ограниченно влияет на систему в целом
Коммуникация	Централизованные HTTP-запросы к одному API	HTTP/gRPC + асинхронные шины и очереди
Тестирование	Проще сквозное тестирование	Требуется отдельно тестировать сервисы и интеграции

Для текущих требований FoodPlanner, объема команды и ресурсов клиент-серверная архитектура обеспечивает оптимальный баланс между скоростью разработки и эксплуатационной устойчивостью, сохраняя возможность дальнейшего масштабирования.

Для дальнейшей детализации архитектуры используется C4-подход, разделяющий описание системы на уровни абстракции:

1. Context — система в окружении пользователей и внешних сервисов;
2. Container — клиент, backend, база данных и внешние интеграции;
3. Component — внутренние модули backend и frontend по доменным областям;
4. Code — уровень классов/модулей и программной реализации.

Проектирование базы данных

При выборе модели хранения данных для FoodPlanner рассматривались основные типы баз данных:

1. реляционные БД — строгая схема, SQL, ACID-транзакции, высокая целостность;
2. документо-ориентированные БД — гибкая схема и высокая адаптивность форматов;
3. key-value и колоночные БД — высокая производительность в специализированных сценариях.

Сравнительная характеристика моделей данных представлена в таблице 5.

Таблица 5 – Сравнительная характеристика моделей баз данных

Критерий	Реляционная модель	Гибкие NoSQL-модели
Структура данных	Таблицы, связи, строгая схема	Документы/ключ-значение/колонки, схема гибкая
Целостность	Высокая за счет ограничений и внешних ключей	Часто контролируется на уровне приложения
Транзакционность	Полная поддержка ACID	Поддержка зависит от конкретной СУБД
Язык запросов	Стандартизированный SQL	Специфичные API и языки запросов
Оптимальный сценарий	Бизнес-системы с богатой связностью данных и требованиями целостности	Высоконагруженные узкоспециализированные сценарии

Для FoodPlanner выбрана реляционная модель с использованием PostgreSQL, так как система оперирует связанной доменной моделью (пользователи, рецепты, планы, отзывы, жалобы, уведомления), для которой критичны целостность, предсказуемость запросов и транзакционная согласованность.

Логическая схема данных отражает ключевые сущности предметной области и связи между ними. Центральными таблицами яв-

ляются users, recipes, ingredients, meal_plans, reviews, complaints, favorites, notifications, а также связующие таблицы recipe_ingredients, meal_plan_items, shopping_list_items.

Внешние ключи, ограничения уникальности и индексы обеспечивают корректность и производительность типовых пользовательских запросов.

Ключевые элементы логической модели представлены ниже фрагментами реальной серверной реализации.

Листинг 1.1 – Модель ролей и пользователей

```
1 class Role(models.Model):
2     class RoleNames(models.TextChoices):
3         USER = "user", "User"
4         MODERATOR = "moderator", "Moderator"
5         ADMIN = "admin", "Admin"

6     name = models.CharField(max_length=32, unique=True,
      ↪  choices=RoleNames.choices)

7 class User(AbstractUser):
8     username = None
9     email = models.EmailField(unique=True)
10    role = models.ForeignKey(Role, on_delete=models.SET_NULL,
      ↪  null=True, blank=True)
11    health_features = models.JSONField(default=list,
      ↪  blank=True)
12    favorite_tags = models.JSONField(default=list,
      ↪  blank=True)
13    USERNAME_FIELD = "email"
```

Листинг 1.2 – Фрагмент модели рецептов и состава ингредиентов

```
1  class Recipe(models.Model):
2      title = models.CharField(max_length=255)
3      category = models.ForeignKey(Category,
       ↪  on_delete=models.SET_NULL, null=True, blank=True)
4      author = models.ForeignKey(settings.AUTH_USER_MODEL,
       ↪  on_delete=models.CASCADE)
5      status = models.CharField(max_length=16,
       ↪  choices=ModerationStatus.choices)
6      tags = models.ManyToManyField(Tag,
       ↪  related_name="recipes", blank=True)

7      class Meta:
8          indexes = [
9              models.Index(fields=["author"]),
10             models.Index(fields=["category"]),
11             models.Index(fields=["status"]),
12         ]

13 class RecipeIngredient(models.Model):
14     recipe = models.ForeignKey(Recipe,
       ↪  on_delete=models.CASCADE,
       ↪  related_name="recipe_ingredients")
15     ingredient = models.ForeignKey(Ingredient,
       ↪  on_delete=models.CASCADE)
16     quantity = models.DecimalField(max_digits=10,
       ↪  decimal_places=2)
```

Листинг 1.3 – Фрагмент модели плана питания и списка покупок

```
1  class MealPlan(models.Model):
2      user = models.ForeignKey(settings.AUTH_USER_MODEL,
       ↪  on_delete=models.CASCADE)
3      start_date = models.DateField()
4      end_date = models.DateField()
5      status = models.CharField(max_length=16,
       ↪  choices=Status.choices, default=Status.DRAFT)

6  class MealPlanItem(models.Model):
7      meal_plan = models.ForeignKey(MealPlan,
       ↪  on_delete=models.CASCADE, related_name="items")
8      day_number = models.PositiveSmallIntegerField()
9      meal_type = models.CharField(max_length=16,
       ↪  choices=MealType.choices)
10     recipe = models.ForeignKey("recipes.Recipe",
       ↪  on_delete=models.CASCADE)

11 class ShoppingList(models.Model):
12     user = models.ForeignKey(settings.AUTH_USER_MODEL,
       ↪  on_delete=models.CASCADE)
13     target_type = models.CharField(max_length=16,
       ↪  choices=TargetType.choices, blank=True, default="")
14     target_id = models.PositiveBigIntegerField(null=True,
       ↪  blank=True)
```

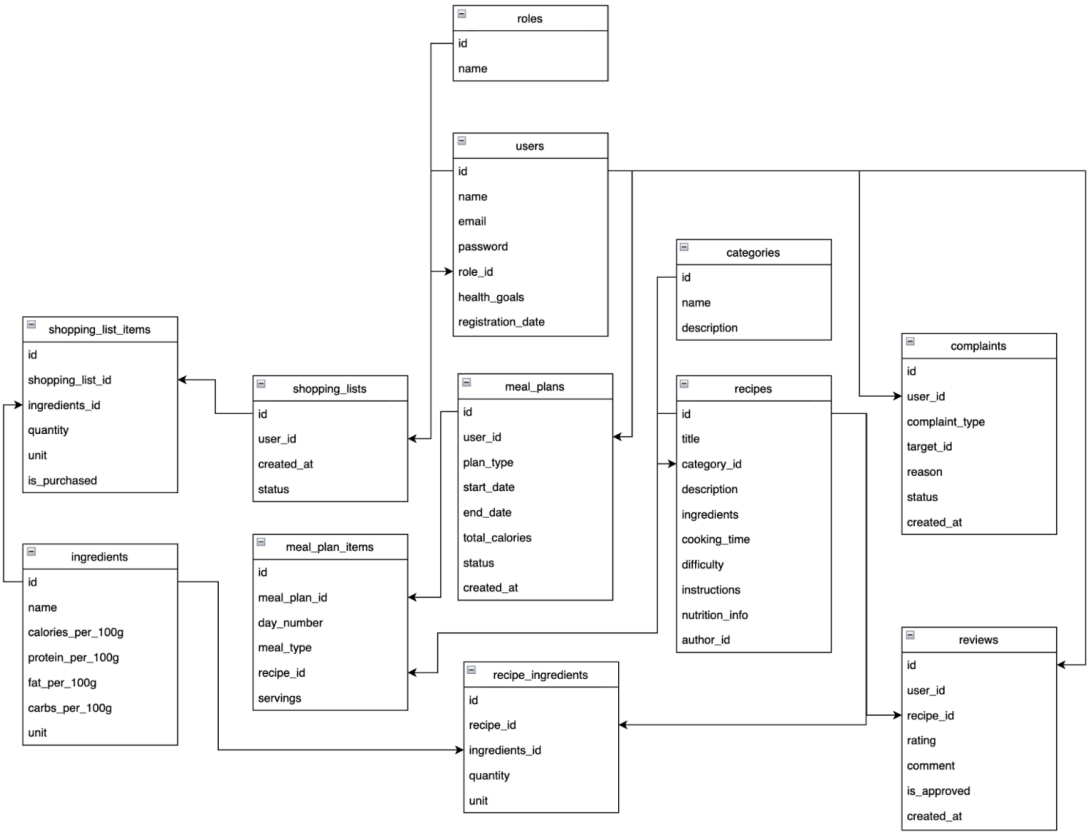

Рисунок 10 – Логическая модель данных FoodPlanner

1.7 Выбор программных и технических средств разработки

Для разработки FoodPlanner использован клиент-серверный подход, поэтому технологический стек разделен на две части: клиентскую (frontend) и серверную (backend). При выборе инструментов учитывались критерии производительности, скорости разработки, удобства сопровождения, зрелости экосистемы и соответствия функциональным требованиям проекта.

Серверная часть реализована на Python с использованием Django и Django REST Framework [25, 26, 27]. В качестве СУБД выбрана PostgreSQL [28]. Для контейнеризации и воспроизводимого развертывания используется Docker и Docker Compose [29, 30].

Для клиентской части выбраны React + TypeScript, управление состоянием выполнено на Effector, UI-слой построен на Mantine, сборка и локальная разработка выполняются через Vite [31, 32, 33, 34, 35].

Сравнительный анализ серверных языков разработки для проекта приведен в таблице 6 с опорой на официальные материалы по языкам и платформам [25, 36, 37, 38].

Таблица 6 – Сравнительный анализ серверных языков

Язык	Преимущества	Ограничения для проекта
Python	Высокая скорость разработки, зрелые веб-фреймворки, низкий порог вхождения	Ниже производительность в CPU-интенсивных задачах по сравнению с компилируемыми языками
Java	Высокая надежность и производительность, зрелая экосистема enterprise-решений	Более высокая сложность и объем кода для быстрого прототипирования
Node.js (TS/JS)	Единый язык для frontend/backend, высокая скорость I/O-сценариев	Требует более строгой дисциплины архитектуры в больших проектах
Go	Высокая производительность и эффективная конкурентность	Меньшая гибкость и более узкая экосистема для сложных административных сценариев

Для прикладных задач FoodPlanner (быстрая реализация доменных модулей, ролевой доступ, модерация, административная логика, API-подход) выбор Python + Django признан оптимальным по балансу трудозатрат и эксплуатационной надежности.

Сравнение выбранных средств клиентской разработки приведено в таблице 7.

Таблица 7 – Выбор средств клиентской разработки

Компонент стека	Обоснование выбора
React + TypeScript	Компонентный подход, типобезопасность и удобная поддержка сложных интерфейсных сценариев
Effector	Детерминированное управление состоянием и понятная модель асинхронных процессов
Mantine	Быстрое построение консистентного UI и сокращение времени реализации типовых паттернов
Vite	Быстрый цикл разработки и сборки SPA-приложения

Таким образом, выбранные программные и технические средства обеспечивают покрытие функциональных требований, поддержку безопасного взаимодействия клиентской и серверной частей, а также возможность поэтапного масштабирования системы.

1.8 Выводы по главе

В первой главе определен проблемный контекст предметной области: пользователи решают взаимосвязанные задачи планирования питания в разрозненных сервисах, что приводит к фрагментации данных и избыточным трудозатратам.

На основании обзора аналогов подтверждено наличие рыночных решений с сильными сторонами, однако выявлены ограничения при реализации комплексного сценария. Это обосновывает разработку собственного веб-приложения, объединяющего полный цикл планирования рациона.

Также сформированы целевая аудитория и ролевая модель, определены функциональные, бизнес- и нефункциональные требования, проведен сравнительный анализ конкурентов и аналогов, описаны бизнес-процессы в IDEF-нотациях и зафиксирована модель данных системы.

2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

2.1 Разработка пользовательского интерфейса приложения

Для реализации пользовательского интерфейса веб-приложения FoodPlanner использовались React и библиотека компонентов Mantine. Выбранный технологический стек позволил сократить время разработки, обеспечить единый визуальный стиль экранов и повысить предсказуемость поведения интерфейса.

Главная страница выполняет функцию стартового навигационного узла и предоставляет пользователю структурированное представление о возможностях платформы. На странице размещены ключевые переходы к тематическим разделам системы: рецептам, планам питания, конструктору, отзывам и личному кабинету. Такое решение обеспечивает быстрое вхождение в систему без необходимости предварительного изучения сложной структуры меню.

Композиционно страница организована как набор контентных секций и карточек предпросмотра. Карточный формат визуально разделяет сущности и помогает пользователю быстро определить целевой сценарий. С точки зрения UX это снижает время поиска нужного действия и повышает вероятность перехода в функциональные разделы. Внешний вид главной страницы приложения представлен на рисунке 11.

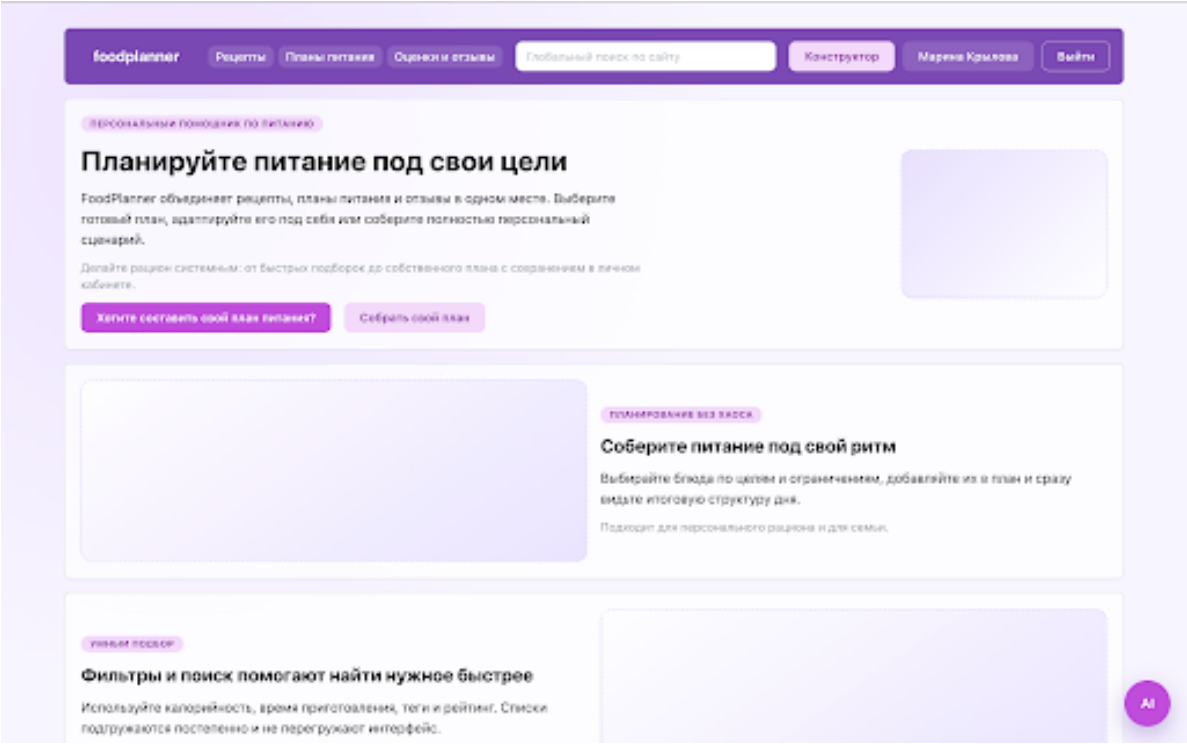

Рисунок 11 – Главная страница приложения

Раздел рецептов реализует каталог с возможностью фильтрации по ключевым параметрам питания и кулинарного процесса. Пользователь может формировать релевантную выборку рецептов по тегам, калорийности, времени приготовления и другим критериям. Использование серверной фильтрации и постраничной загрузки поддерживает высокую отзывчивость интерфейса при росте объема данных.

Дополнительно реализована логика корректного отображения элементов пагинации: кнопка «Показать еще» отображается только при наличии данных для следующей подгрузки. Карточки рецептов содержат ключевые атрибуты для первичного выбора: название, краткое описание, теги, время приготовления и базовые показатели. Интерфейс раздела рецептов представлен на рисунке 12.

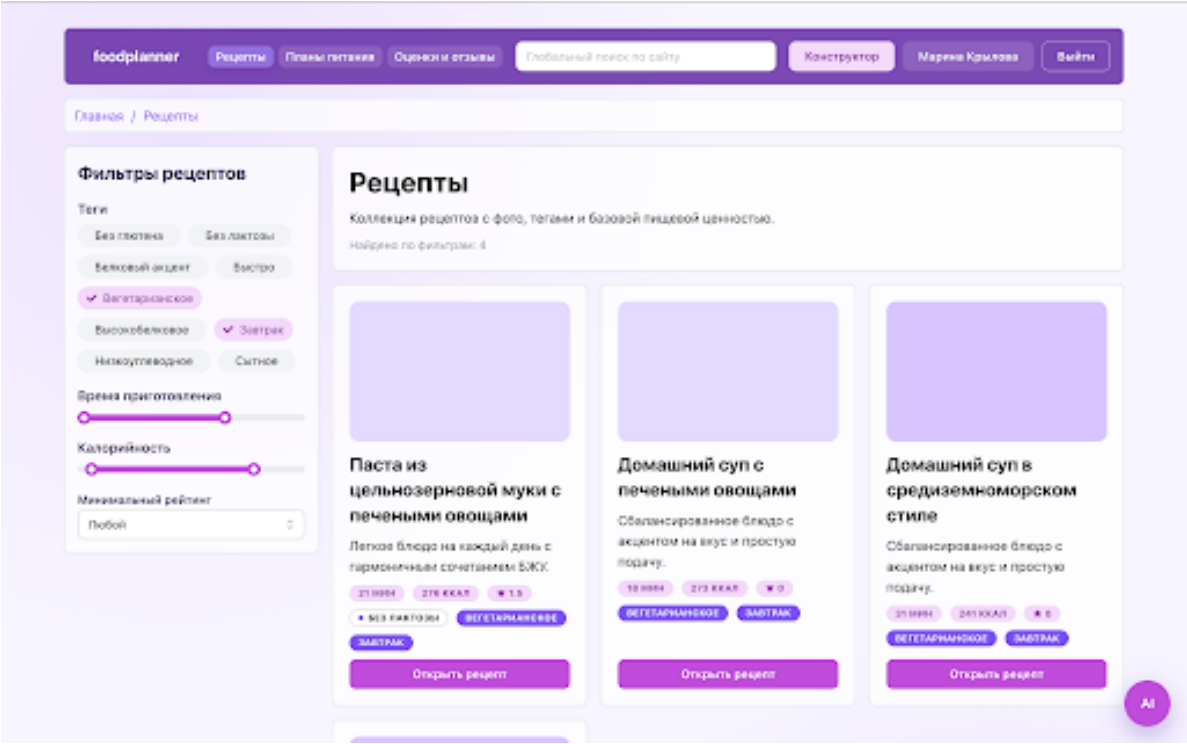

Рисунок 12 – Страница рецептов

Детальная страница рецепта объединяет информационный и прикладной контуры. Информационный блок включает название, описание, пищевую ценность (калории, белки, жиры, углеводы), теги и пошаговую инструкцию. Прикладной блок реализует интерактивные действия: управление ингредиентами, формирование списка покупок, сохранение в профиль и экспорт.

Особое значение имеет блок ингредиентов с чекбоксами: пользователь отмечает наличие позиций, после чего система формирует список недостающих покупок. При повторном посещении страницы состояние чекбоксов восстанавливается, что поддерживает непрерывность пользовательского сценария. Детальная страница рецепта представлена на рисунке 13.

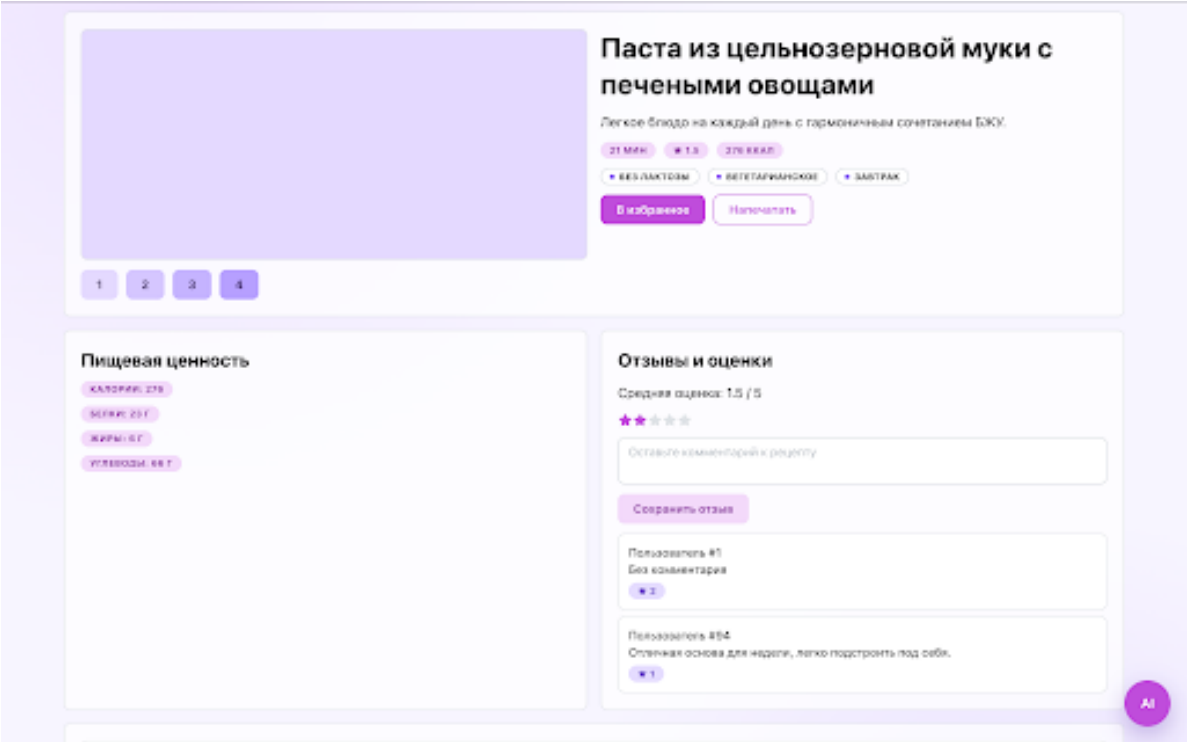

Рисунок 13 – Страница конкретного рецепта

Раздел планов питания является витриной готовых рационов и ориентирован на задачу быстрого выбора стратегии питания. Пользователь получает список планов с фильтрацией по типу, калорийности, рейтингу и дополнительным параметрам, включая персонализированные предпочтения. Серверная пагинация позволяет обрабатывать большие массивы данных без перегрузки интерфейса.

Карточка плана содержит ключевые характеристики для принятия решения: автор, тип плана, длительность, агрегированные показатели и оценка. Унификация карточек между разделами сокращает порог обучения и обеспечивает консистентный пользовательский опыт. Интерфейс страницы планов питания представлен на рисунке 14.

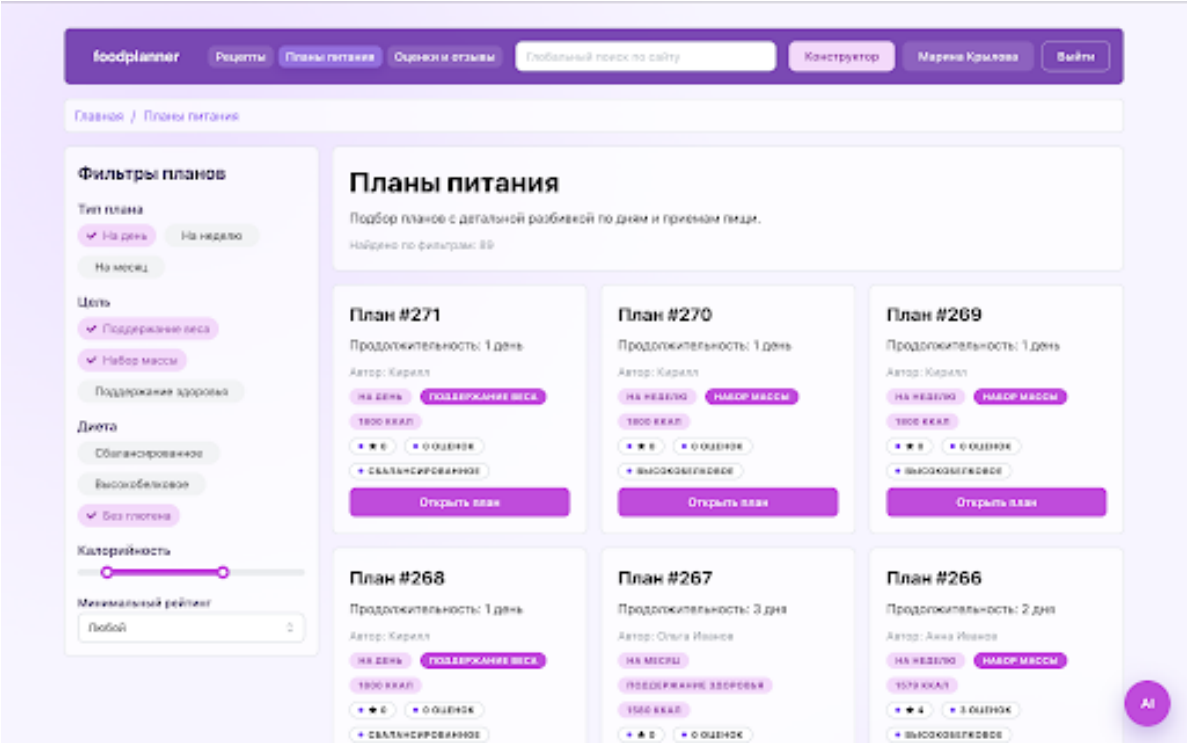

Рисунок 14 – Страница планов питания

Детальная страница плана питания раскрывает структуру рациона по дням и приемам пищи (завтрак, обед, ужин, перекусы) с привязкой к рецептам и пищевым параметрам. Такой формат позволяет оценить рацион не только по агрегированным показателям, но и на уровне практического применения по каждому дню.

Интерфейс поддерживает раскрывающиеся секции и группировку данных, что упрощает навигацию внутри длинных планов и снижает визуальный шум. Как и на странице рецепта, доступны действия прикладного уровня: работа с ингредиентами, формирование списка покупок и сохранение результатов. Детальная страница плана питания представлена на рисунке 15.

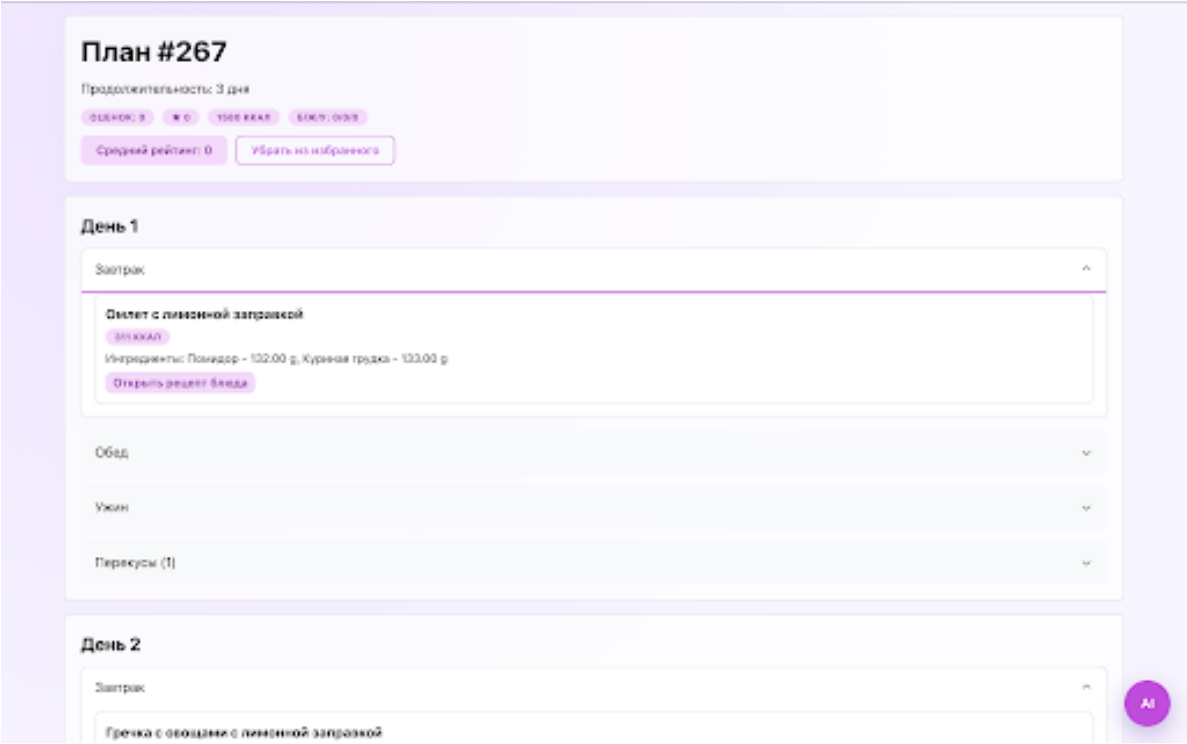

Рисунок 15 – Страница конкретного плана питания

Конструктор является центральным функциональным модулем приложения и предназначен для создания персонализированных планов питания. Интерфейс позволяет задавать параметры плана, выбирать рецепты из каталога и распределять их по календарной структуре. Реализована визуальная сегментация дней и поддержка вариативного количества перекусов.

Пользователь одновременно видит доступный контент и текущую структуру плана, что упрощает составление и корректировку рациона. По завершении формирования план отправляется на модерацию, после чего рабочая область конструктора подготавливается к следующему сценарию. Интерфейс конструктора представлен на рисунке 16.

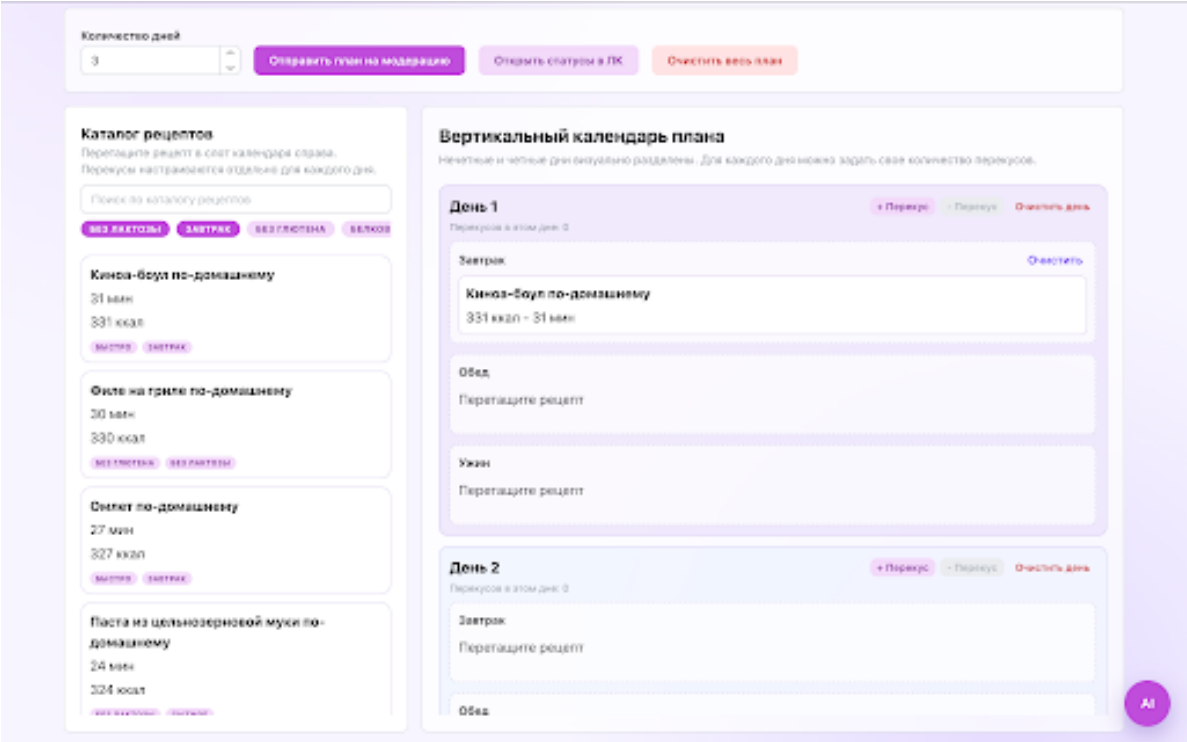

Рисунок 16 – Страница конструктора плана питания

Кабинет модератора реализован как специализированный интерфейс принятия решений. Для модератора скрыты нерелевантные персональные блоки, а в фокус вынесены заявки на проверку. Это снижает информационный шум и поддерживает операционный характер работы.

В блоке модерации рецептов отображаются карточки материалов в статусе проверки с ключевыми метаданными: тип сущности, автор, дата подачи и название. Каждая карточка предоставляет действия одобрения/отклонения и переход к детальному просмотру. Интерфейс кабинета модератора представлен на рисунке 17.

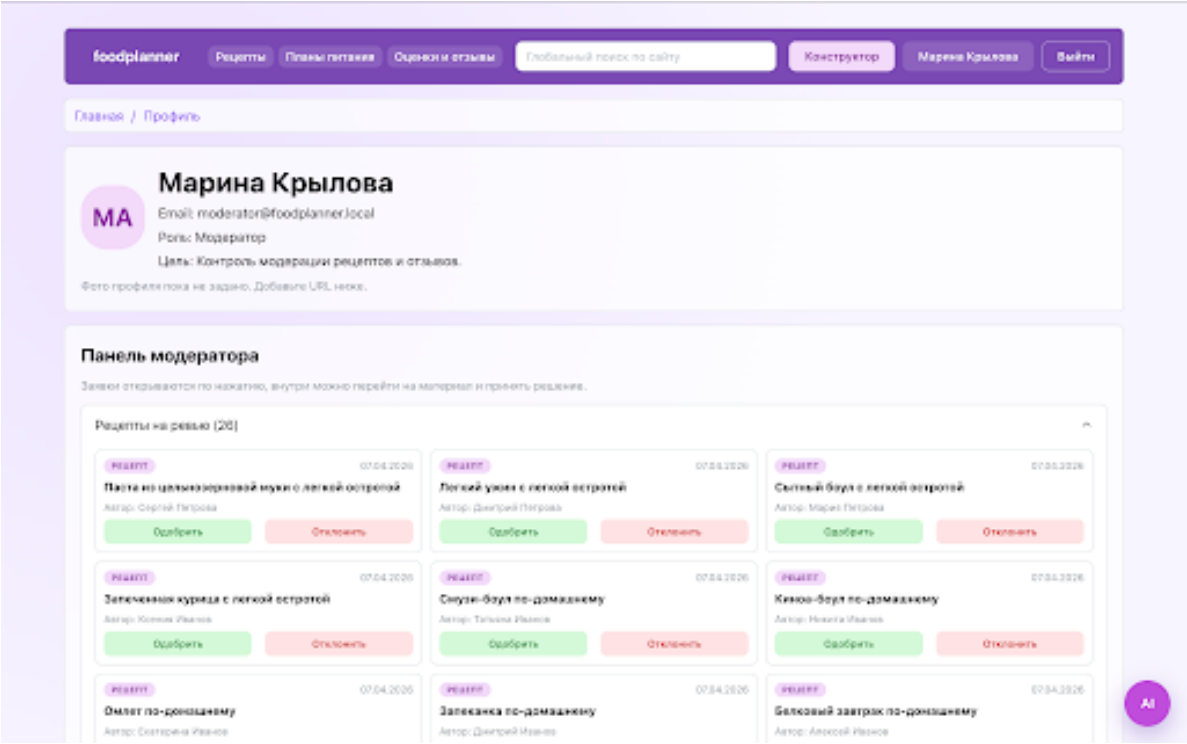

Рисунок 17 – Личный кабинет модератора с модерацией

Личный кабинет пользователя предназначен для управления персональными данными и параметрами питания, влияющими на персонализацию контента. Пользователь может обновлять профиль, указывать цели, предпочтительную диету, особенности питания и любимые теги.

С точки зрения UX профиль построен как набор самостоятельных функциональных блоков: данные аккаунта, персональные предпочтения, избранное, собственные материалы, статусы модерации и связанные сущности (включая списки покупок). Поддержка пустых состояний и уведомлений о результате операций обеспечивает понятную обратную связь пользователю. Внешний вид личного кабинета пользователя представлен на рисунках 18 и 19.

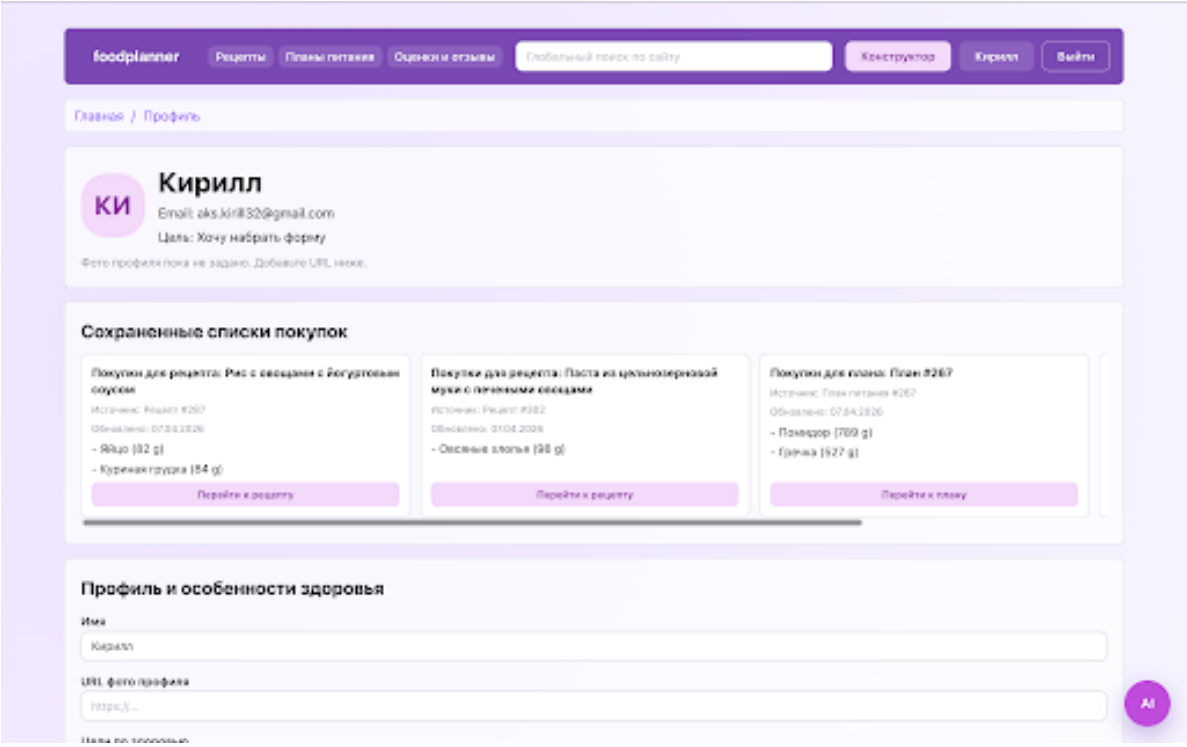

Рисунок 18 – Личный кабинет пользователя

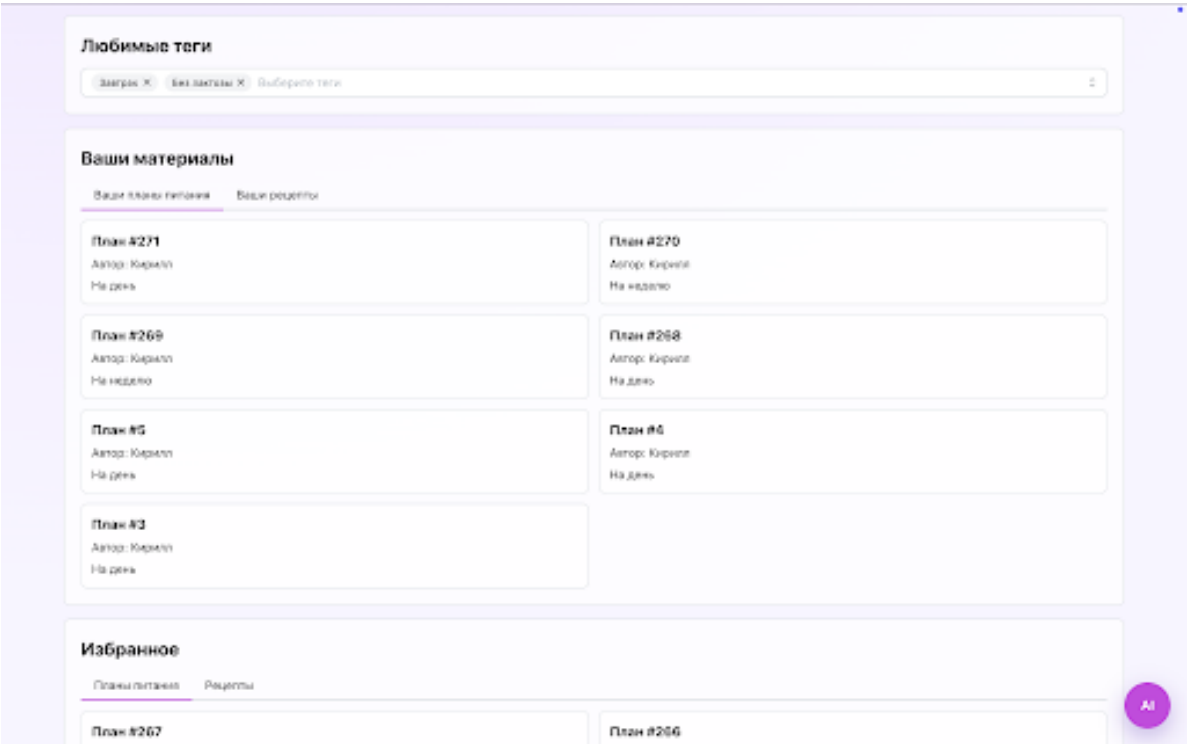

Рисунок 19 – Личный кабинет пользователя

ИИ-ассистент интегрирован в приложение как доступный с любого экрана диалоговый интерфейс. Пользователь может открыть чат через плавающую кнопку и получить консультацию по рациону, подбору блюд и общим вопросам питания. Реализация в формате выдвижной панели обеспечи-

вает быстрый доступ к функции без прерывания основного пользовательского сценария.

Интерфейс чата включает историю сообщений, поле ввода с повышенной читаемостью и явные состояния отправки/ожидания ответа. Дополнительно реализовано форматирование длинных ответов и обработка системных ошибок внешнего API через уведомления. Открытое диалоговое окно с ИИ-ассистентом представлено на рисунке 20.

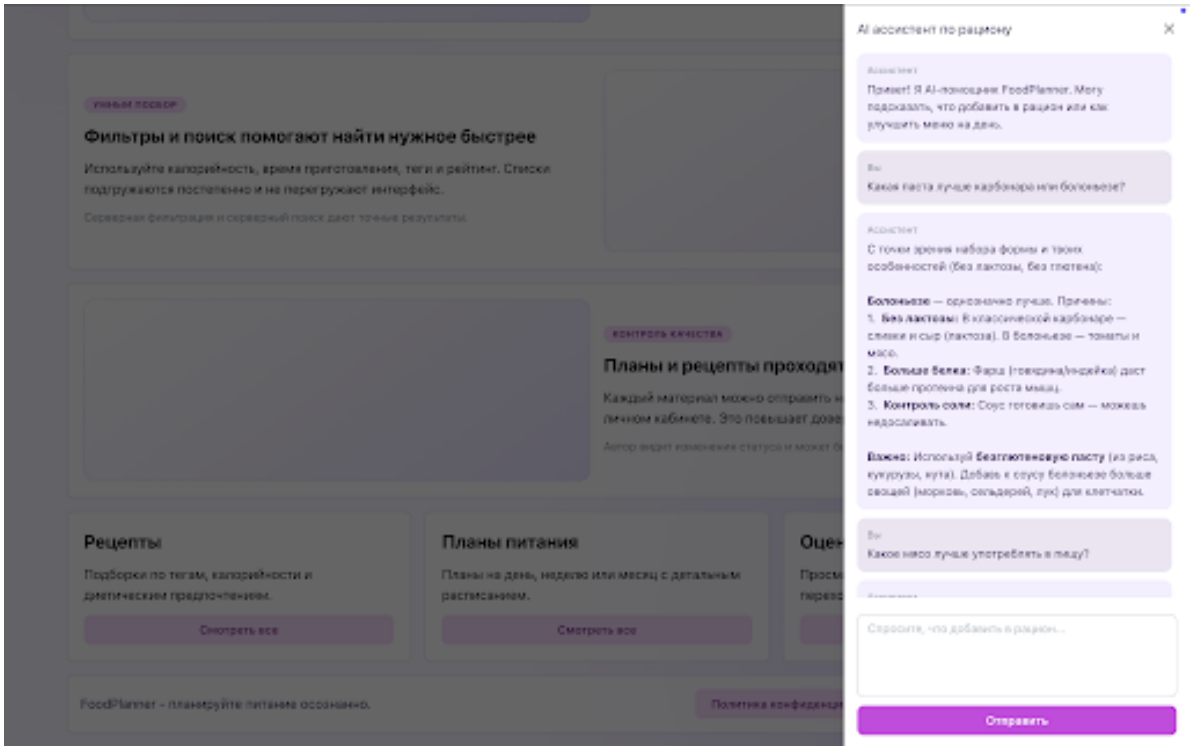

Рисунок 20 – Открытое диалоговое окно с ИИ-ассистентом

Раздел оценок и отзывов обеспечивает прозрачный механизм социальной валидации контента. Пользователь может анализировать оценки и текстовые комментарии других участников платформы, что повышает доверие к опубликованным материалам и помогает принимать обоснованные решения при выборе рецептов и планов питания.

Функционально раздел агрегирует пользовательский опыт и одновременно является каналом обратной связи для авторов контента и модераторов. Интерфейс экрана реализован в формате карточек и списков с акцентом на читаемость комментариев, рейтинг и связь с объектом оценки. Страница оценок и отзывов представлена на рисунке 21.

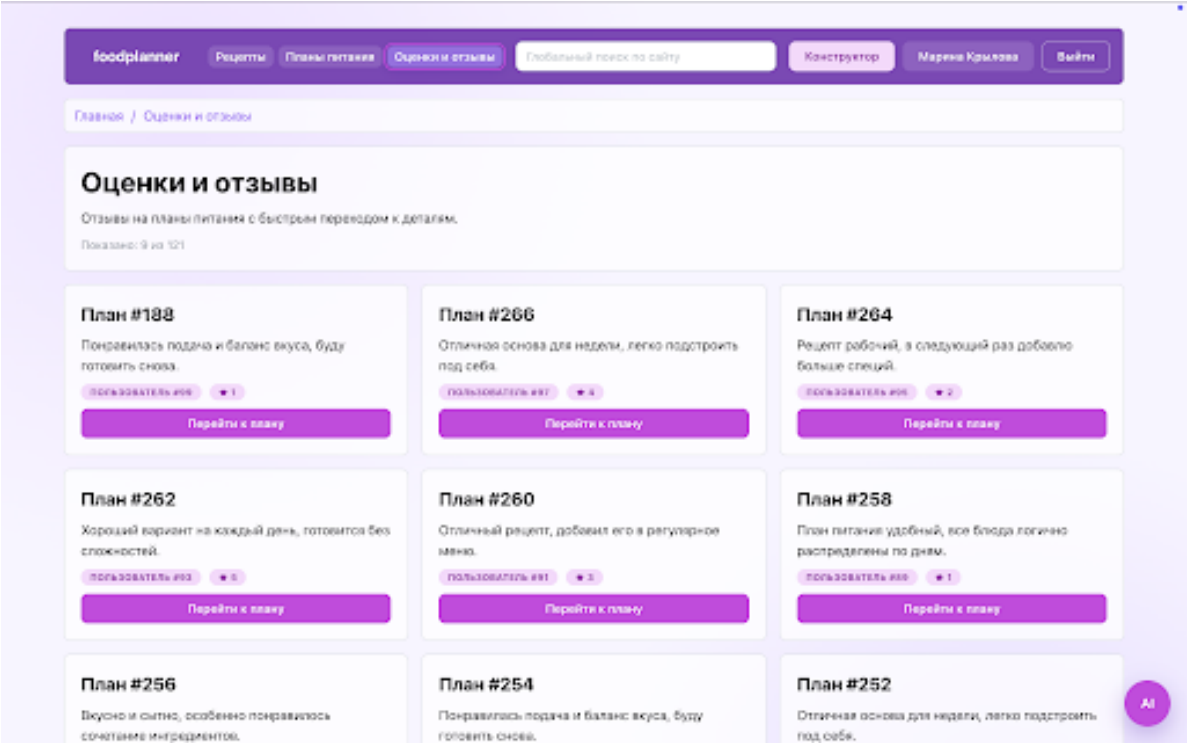

Рисунок 21 – Страница оценок и отзывов

2.2 Техническая реализация серверной части

Информация о выборе архитектурного подхода и программных средств приведена в разделе 1.7. В данном разделе рассматриваются практические аспекты непосредственной серверной разработки: структура проекта, организация слоев ответственности, работа с базой данных, реализация API и поддержка ключевых бизнес-процессов.

Серверная часть FoodPlanner реализована на языке Python с использованием Django и Django REST Framework [25, 26, 27]. Такой выбор позволил реализовать прикладную логику в сжатые сроки и при этом сохранить высокий уровень структурированности кода. В качестве основной СУБД используется PostgreSQL [28]; для локальной отладки и быстрых проверок может использоваться SQLite.

Структура директории проекта серверной части. Серверный код размещен в директории `Backend` и организован по доменным приложениям. Каждое приложение изолирует отдельную часть предметной области и име-

ет собственные модели, сериализаторы, представления и маршруты. Ниже приведено назначение ключевых директорий:

1. `/Backend` — корневая директория серверной части, содержащая конфигурацию проекта, доменные модули, миграции и инфраструктурные файлы;

2. `/Backend/config` — модуль глобальной конфигурации: параметры Django, middleware, CORS/CSRF, подключение приложений, верхнеуровневые URL и служебные endpoint'ы;

3. `/Backend/users` — модуль пользователей и ролей: регистрация, вход/выход, профиль, ролевой доступ;

4. `/Backend/recipes` — модуль рецептов и справочников: категории, теги, ингредиенты, рецепты, фильтрация и модерационные действия;

5. `/Backend/meal_plans` — модуль планов питания и shopping list: создание, редактирование, отправка на модерацию, детализация по дням и приемам пищи;

6. `/Backend/interactions` — модуль пользовательских взаимодействий: отзывы, оценки, жалобы, избранное;

7. `/Backend/notifications` — модуль уведомлений по событиям платформы;

8. `/Backend/moderation` — модуль журналирования модерационных действий;

9. `/Backend/assistant` — модуль ИИ-ассистента: серверный прокси-запрос к внешнему LLM-провайдеру с обработкой ошибок и контекста;

10. `/Backend/*/migrations` — миграции схемы БД для каждого доменного приложения;

11. `/Backend/users/management/commands` — management-команды (в том числе `seed_demo_data`) для ускорения тестирования сценариев.

Организация серверной части по указанным модулям обеспечивает разделение ответственности, упрощает тестирование и снижает стоимость последующих изменений. Структура серверной части представлена на рисунке 22.

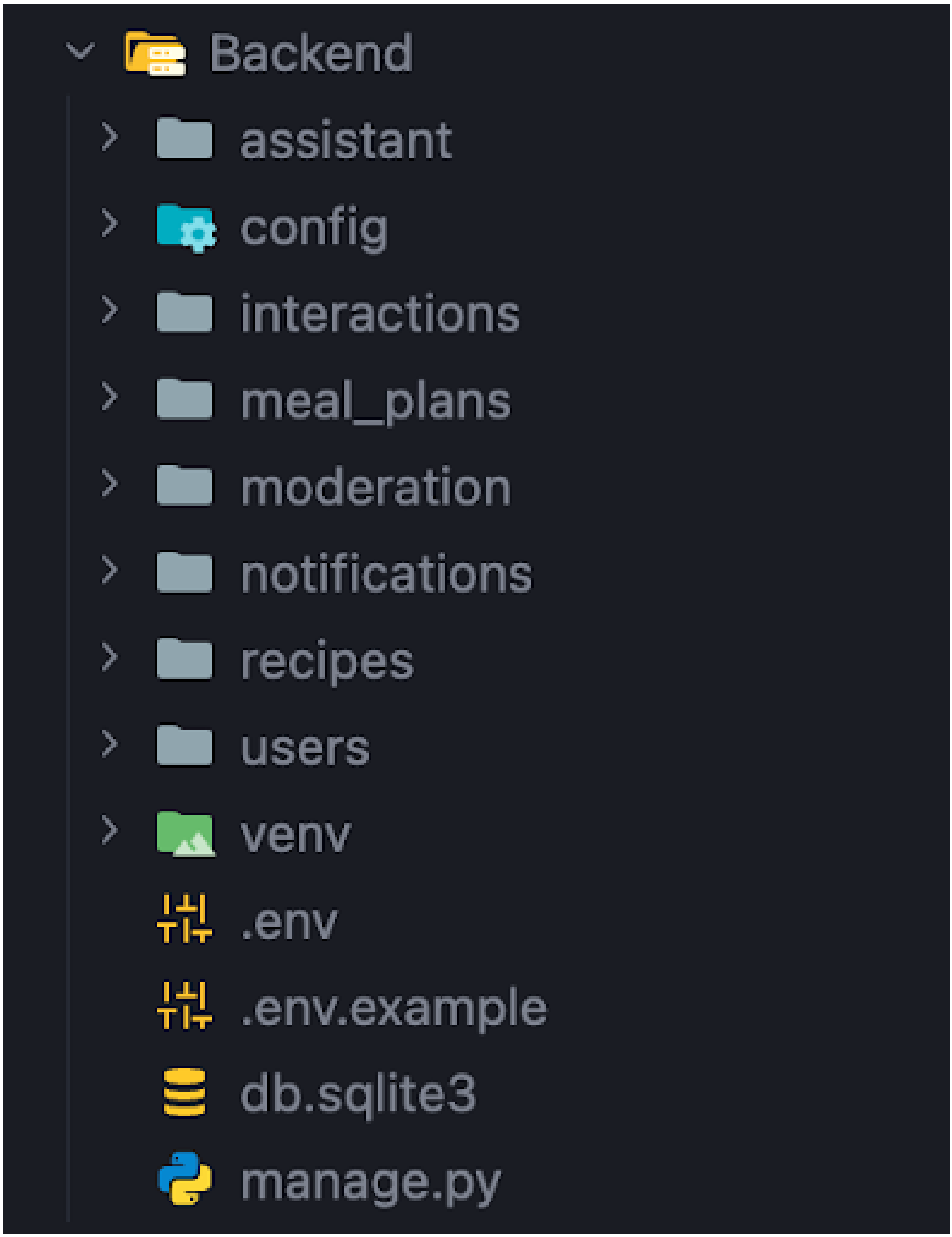

Рисунок 22 – Структура серверной части проекта

Слойная организация серверной логики. Серверная часть построена по принципу разделения ответственности между тремя уровнями:

1. уровень API (views/viewsets + serializers) — прием входящих запросов, валидация формата данных, маршрутизация, формирование ответов;
2. уровень бизнес-логики (доменные правила в приложениях, permissions, actions) — реализация прикладных сценариев, проверка прав доступа, обработка статусов модерации и переходов жизненного цикла сущностей;
3. уровень данных (models + ORM + migrations) — описание структуры БД, связей, ограничений целостности, индексов и миграций.

Такая модель не допускает концентрации всей логики в одном слое: API-уровень отвечает за протокол взаимодействия, доменный уровень — за поведение системы, а модельный уровень — за целостность хранения данных.

Реализация API и ключевые серверные сценарии. Сервер предоставляет REST API для всех основных ролей и пользовательских сценариев платформы. Ключевые контуры API включают:

1. пользователей и профиль (регистрация, авторизация, выход, чтение и обновление профиля);
2. рецепты (каталог, фильтрация, поиск, детальная карточка, модерация);
3. планы питания (создание, редактирование, отправка на модерацию, решение модератора);
4. отзывы и оценки (создание, изменение, модерация);
5. жалобы и избранное (подача и обработка жалоб, пользовательские коллекции);
6. списки покупок (формирование из рецепта/плана и сохранение в профиль);
7. уведомления по событиям модерации и пользовательским действиям;

8. ИИ-ассистент через безопасный backend-proxy endpoint.

Работа с базой данных и моделью данных. В рассматриваемом проекте взаимодействие с БД реализовано преимущественно через Django ORM. В отличие от подхода с полностью ручным формированием SQL-запросов, ORM-подход был выбран осознанно и обусловлен следующими факторами:

1. повышение скорости и надежности разработки за счет типовых операций чтения/записи без дублирования SQL-кода;
2. единообразие и предсказуемость доступа к данным во всех доменных модулях проекта;
3. снижение вероятности ошибок в CRUD-операциях и упрощение сопровождения при изменении схемы.

Использование ORM в проекте не исключает контроля производительности. Для ресурсоемких сценариев применяются индексы, ограничения целостности, оптимизация выборок через механизмы ORM и явное проектирование связей между сущностями. Такая комбинация позволяет сохранить баланс между удобством разработки и эффективностью выполнения запросов.

Модель данных включает сущности пользователей, ролей, рецептов, ингредиентов, планов питания, элементов плана, отзывов, жалоб, избранного, уведомлений, списков покупок и журналов модерации. Между сущностями используются связи «один-ко-многим» и «многие-ко-многим», что позволяет корректно покрывать прикладные сценарии.

Для критичных направлений (поиск, модерация, привязка shopping list к источнику, контроль уникальности взаимодействий) введены ограничения целостности и индексы. Миграции фиксируют эволюцию схемы БД и обеспечивают воспроизводимое развертывание на разных окружениях.

Хранение прикладных данных и медиа. Основные транзакционные данные (профили, рецепты, планы, отзывы, жалобы, уведомления) сохраняются в PostgreSQL. Файловые ресурсы в текущей версии могут размещаться в файловой системе сервера, что достаточно для учебного и пилотного конту-

ра эксплуатации. При переходе к промышленной эксплуатации архитектура допускает перенос медиа-ресурсов в объектное S3-совместимое хранилище без изменения прикладной бизнес-логики, за счет абстракции слоя хранения.

Пример получения пользователя при авторизации через сериализатор DRF приведен в листинге 2.1. В отличие от ручного сравнения паролей в SQL-запросе, в проекте используется встроенный механизм `authenticate`, который учитывает хеширование паролей и backend-аутентификации Django.

Листинг 2.1 – Проверка учетных данных пользователя при входе

```
1  class LoginSerializer(serializers.Serializer):
2      email = serializers.EmailField()
3      password = serializers.CharField(write_only=True)

4      def validate(self, attrs):
5          user = authenticate(
6              request=self.context.get("request"),
7              username=attrs["email"],
8              password=attrs["password"],
9          )
10         if not user:
11             raise serializers.ValidationError("Invalid
               ↪  credentials.")
12         attrs["user"] = user
13         return attrs
```

Конфигурация подключения к базе данных определяется централизованно через настройки приложения и переменные окружения (листинг 2.2). Такой подход выполняет ту же функцию, что и ручная singleton-инициализация подключения в других стеках, но в терминах Django и его жизненного цикла.

Листинг 2.2 – Конфигурация подключения к PostgreSQL в Django

```
1  if env("POSTGRES_DB", default=""):
2      DATABASES = {
3          "default": {
4              "ENGINE": "django.db.backends.postgresql",
5              "NAME": env("POSTGRES_DB"),
6              "USER": env("POSTGRES_USER", default="postgres"),
7              "PASSWORD": env("POSTGRES_PASSWORD",
               ↪  default="postgres"),
8              "HOST": env("POSTGRES_HOST",
               ↪  default="localhost"),
9              "PORT": env("POSTGRES_PORT", default="5432"),
10         }
11     }
12 else:
13     DATABASES = {
14         "default": {
15             "ENGINE": "django.db.backends.sqlite3",
16             "NAME": BASE_DIR / "db.sqlite3",
17         }
18     }
```

Использование переменных окружения является обязательной частью конфигурации backend [39]. Переменные хранят параметры подключения, секретные ключи и настройки окружения без их жесткого встраивания в исходный код. Это повышает безопасность и позволяет использовать различные значения для разработки и production-среды (листинг 2.3).

Листинг 2.3 – Загрузка переменных окружения в настройках backend

```
import environ

BASE_DIR = Path(__file__).resolve().parent.parent
env = environ.Env(
    DEBUG=(bool, True),
    SECRET_KEY=(str, "django-insecure-change-me"),
)
environ.Env.read_env(BASE_DIR / ".env")

DEBUG = env("DEBUG")
SECRET_KEY = env("SECRET_KEY")
```

Персонализация и серверная фильтрация. Одной из ключевых задач backend является выдача релевантных рецептов и планов питания с учетом параметров профиля (цели, теги, особенности питания). Фильтрация и пагинация реализованы на сервере, что обеспечивает:

1. корректную обработку больших выборок;
2. снижение нагрузки на клиент;
3. единый источник правды по составу результатов;
4. стабильное поведение интерфейса при комбинировании фильтров.

Модерация как самостоятельный процесс. Модерационный контур реализован отдельно от пользовательского потока: пользователь создает контент и отправляет его на проверку, модератор или администратор принимает решение, после чего статус сущности обновляется и пользователю направляется уведомление. Такой подход обеспечивает управляемый жизненный цикл контента, аудит действий и масштабируемость процесса при росте нагрузки.

Используемые внешние библиотеки. Практическая реализация серверной части опирается на следующие группы библиотек [26, 27, 40, 41, 24, 39, 42]:

1. Django/DRF — веб-фреймворк и REST-интерфейс;
2. psycopg — драйвер подключения к PostgreSQL;

3. drf-spectacular — генерация OpenAPI/Swagger-схемы;
4. django-cors-headers — управление CORS-политиками;
5. django-environ — загрузка конфигурации из переменных окружения;
6. gunicorn — production WSGI-сервер;
7. стандартные модули Python для сетевого взаимодействия с внешним API ИИ-ассистента.

Описание конечных точек API. Взаимодействие между клиентской и серверной частями реализовано в стиле REST с использованием протокола HTTP [43, 44]. Обмен данными строится на стандартных методах:

1. `GET` — получение ресурса или коллекции ресурсов;
2. `POST` — создание ресурса или запуск прикладного действия;
3. `PUT/PATCH` — полное или частичное обновление существующего ресурса;
4. `DELETE` — удаление ресурса (или логическое удаление в рамках бизнес-логики).

Каждой группе ресурсов соответствует набор endpoint'ов, объединенных общим префиксом `/api`. Для отладки и тестирования API на этапе разработки использовались Swagger-интерфейс (`/api/docs/`) и ручная проверка запросов в Insomnia/Postman [45, 46, 47].

Список ключевых endpoint'ов серверной части:

1. аутентификация и профиль пользователя: `POST /api/users/register/`, `POST /api/users/login/`, `POST /api/users/logout/`, `GET/PATCH /api/users/me/`;
2. администрирование пользователей: `GET/POST /api/users/manage/`, `GET/PATCH/DELETE /api/users/manage/{id}/`;
3. рецепты и справочники: `GET/POST /api/recipes/`, `GET/PATCH/DELETE /api/recipes/{id}/`, `POST /api/recipes/{id}/moderate/`, `GET`

/api/recipes/categories/, GET /api/recipes/ingredients/, GET /api/recipes/tags/;

4. планы питания и списки покупок: GET/POST /api/meal-plans/, GET/PATCH/DELETE /api/meal-plans/{id}/, POST /api/meal-plans/{id}/submit_for_m POST /api/meal-plans/{id}/moderate/, GET/POST /api/meal-plans/shopping-lists/, POST /api/meal-plans/shopping-lists/sync-target/;

5. пользовательские взаимодействия: GET/POST /api/interactions/reviews/, POST /api/interactions/reviews/{i GET/POST /api/interactions/complaints/, POST /api/interactions/complaints/{id}/resolve/, GET/POST /api/interactions/favorites/;

6. уведомления и ассистент: GET /api/notifications/, POST /api/notifications/{id}/mark_read/, POST /api/assistant/chat/;

7. служебные endpoint'ы: GET /api/health/, GET /api/schema/, GET /api/docs/.

2.3 Техническая реализация клиентской части

Клиентская часть FoodPlanner реализована как одностраничное веб-приложение (SPA) на React + TypeScript [31, 32]. Выбор данного подхода обусловлен необходимостью обеспечить быстрый отклик интерфейса, типобезопасность при работе с данными API и удобство развития проекта при добавлении новых пользовательских сценариев.

В качестве UI-библиотеки используется Mantine, которая предоставляет готовые адаптивные компоненты и упрощает поддержку единого визуального стиля [34]. Для управления состоянием применен Effector, позволяющий явно описывать события, эффекты и зависимости между состояниями [33].

Структура проекта клиентской части. Код приложения организован по модульному принципу, где каждый каталог отвечает за ограниченную область ответственности:

1. `/Frontend/src/pages` — страницы верхнего уровня (маршруты пользовательского интерфейса);

2. `/Frontend/src/components` — переиспользуемые компоненты каркаса и представления;

3. `/Frontend/src/features` — feature-модули со сценарием и состоянием (например, аутентификация, профиль, конструктор плана);

4. `/Frontend/src/shared/api` — централизованный слой HTTP-взаимодействия с backend и обработка ошибок;

5. `/Frontend/src/shared/ui` — общие UI-блоки (уведомления, виджет ассистента, состояния загрузки/пустых данных);

6. `/Frontend/src/shared/model` — общие клиентские модели и события;

7. `/Frontend/src/types` — доменные типы и DTO-представления данных.

Структура проекта клиентской части представлена на рисунке 23.

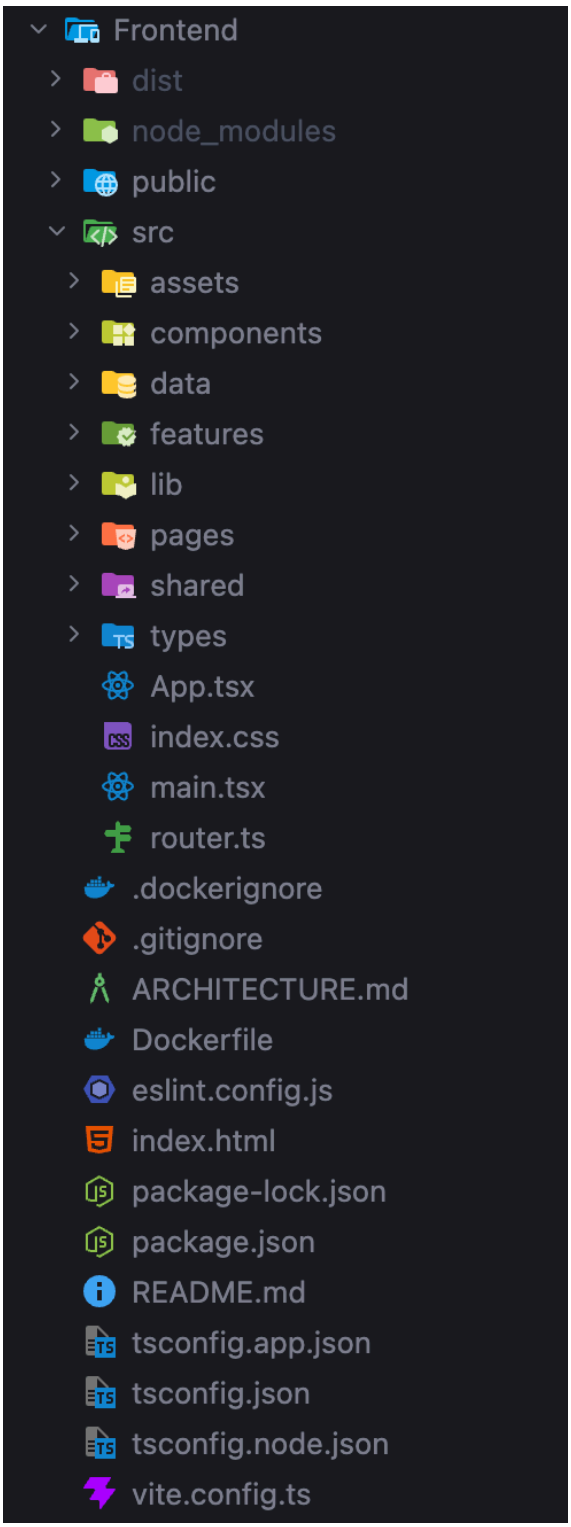

Рисунок 23 – Структура проекта клиентской части

Слойная архитектура клиентского приложения включает:

1. presentation layer — React-страницы и компоненты визуализации;
2. state layer — модели Effector и локальные состояния интерфейса;
3. data access layer — API-функции, инкапсулирующие HTTP-запросы, CSRF-защиту, преобразование DTO и обработку ошибок.

Разработка экранов и навигации. Экранная структура клиентской части соответствует пользовательским сценариям, рассмотренным в разделе 2.1: каталог рецептов, карточка рецепта, планы питания, конструктор, отзывы, профиль и авторизация. Для маршрутизации используется `react-router-dom` [48]; базовый набор маршрутов приложения приведен в листинге 2.4.

Листинг 2.4 – Маршрутизация экранов клиентской части

```
1  <Routes>
2    <Route path="/" element={<AboutPage />} />
3    <Route path="/auth" element={<AuthPage />} />
4    <Route path="/recipes" element={<RecipesPage />} />
5    <Route path="/recipes/:recipeId"
     ↪  element={<RecipeDetailPage />} />
6    <Route path="/meal-plans" element={<MealPlansPage />} />
7    <Route path="/meal-plans/:planId"
     ↪  element={<MealPlanDetailPage />} />
8    <Route path="/reviews" element={<ReviewsPage />} />
9    <Route path="/planner" element={<PlannerPage />} />
10   <Route path="/profile" element={<ProfilePage />} />
11   <Route path="*" element={<FallbackCard message="Страница не
     ↪  найдена." />} />
12 </Routes>
```

Управление состоянием в клиентской части реализовано через Effector. Для аутентификации используются события пользовательских действий, асинхронные эффекты и производные состояния статуса сессии. Пример модели авторизации приведен в листинге 2.5.

Листинг 2.5 – Модель состояния аутентификации на Effector

```
1 export const checkSessionRequested = createEvent()
2 export const loginSubmitted = createEvent<LoginPayload>()
3 export const logoutRequested = createEvent()

4 export const checkSessionFx = createEffect(async () =>
  ↪ getMe())
5 export const loginFx = createEffect(async (payload:
  ↪ LoginPayload) => login(payload))
6 export const logoutFx = createEffect(async () => logout())

7 export const $authStatus =
  ↪ createStore<AuthStatus>('checking')
8   .on(checkSessionFx.done, () => 'auth')
9   .on(checkSessionFx.fail, () => 'guest')
10  .on(loginFx.done, () => 'auth')
11  .on(logoutFx.done, () => 'guest')

12 sample({ clock: checkSessionRequested, target:
  ↪ checkSessionFx })
13 sample({ clock: loginSubmitted, target: loginFx })
14 sample({ clock: logoutRequested, target: logoutFx })
```

Интеграция с серверной частью построена через централизованный API-слой с единым форматом вызовов `GET/POST/PATCH/DELETE`. Для state-changing запросов используется CSRF-токен, а также включен режим `credentials: include` для корректной работы с сессионной аутентификацией [21, 26]. Фрагмент клиентского API-слоя приведен в листинге 2.6.

Листинг 2.6 – Клиентский API-слой с CSRF-защитой

```
1 export async function ensureCsrfCookie() {
2   if (getCookie('csrftoken')) return
3   await fetch(`${API_BASE_URL}/health/`, { credentials:
    ↪  'include' })
4 }

5 async function apiRequest<T>(path: string, method: 'GET' |
  ↪  'POST' | 'PATCH', body?: unknown) {
6   if (method !== 'GET') await ensureCsrfCookie()
7   const csrfToken = getCookie('csrftoken')

8   const response = await fetch(`${API_BASE_URL}${path}`, {
9     method,
10    credentials: 'include',
11    headers: method === 'GET' ? undefined : {
12      'Content-Type': 'application/json',
13      'X-CSRFToken': csrfToken,
14    },
15    body: body ? JSON.stringify(body) : undefined,
16  })

17  if (!response.ok) throw await toApiError(response)
18  return response.status === 204 ? (undefined as T) :
    ↪  ((await response.json()) as T)
19 }
```

Дополнительным пользовательским сценарием является работа с ИИ-ассистентом: в интерфейсе реализован плавающий виджет с открытием боковой панели, сохранением истории диалога, отправкой сообщений через backend-proxy endpoint и обработкой ошибок с выводом уведомлений.

2.4 Вывод

Во второй главе рассмотрены практические аспекты технической реализации системы FoodPlanner. Описаны интерфейсные решения, серверная архитектура, организация клиентского и серверного кода, а также ключевые принципы интеграции между слоями приложения. Представленные решения обеспечивают функциональную полноту пользовательских сценариев и создают устойчивую основу для дальнейшего развития платформы.

3 ТЕСТИРОВАНИЕ, РАЗВЕРТЫВАНИЕ И ВНЕДРЕНИЕ СИСТЕМЫ

3.1 Тестирование системы

Тестирование выполнено для подтверждения соответствия реализованного решения функциональным и нефункциональным требованиям, сформулированным в главе 1. Проверка включала серверную часть, клиентский интерфейс и интеграционные пользовательские сценарии.

Тестирование серверной части

Для backend-части (Django + DRF) использован модульный подход [26, 27]:

1. проверка корректности доменных моделей и ограничений целостности;
2. проверка прав доступа по ролевой модели `user/moderator/admin`;
3. проверка сценариев модерации, обработки жалоб и уведомлений;
4. проверка валидации входных данных и обработки ошибок API.

Критериями приемки серверной части являются:

1. успешное применение миграций и отсутствие конфликтов схемы БД;
2. корректная работа основных API-методов (CRUD и служебные actions);
3. отсутствие критических ошибок в системной проверке Django.

Тестирование клиентской части

Для frontend-части (React + Effector) проведено функциональное тестирование ключевых пользовательских потоков [31, 33]:

1. авторизация и работа с профилем пользователя;
2. просмотр и фильтрация каталогов рецептов и планов;
3. формирование плана питания в конструкторе;
4. работа с отзывами, избранным и списками покупок;
5. обработка ошибок запросов и отображение уведомлений.

Отдельно проверены граничные условия конструктора плана питания:

1. длительность плана задается в диапазоне от 1 до 30 дней;
2. шаг изменения длительности составляет 1 день;
3. количество перекусов задается в диапазоне от 0 до 5 в сутки.

Интеграционное тестирование

Интеграционная проверка выполнена на уровне бизнес-сценариев:

1. регистрация пользователя и заполнение профиля;
2. создание рецепта, отправка на модерацию, изменение статуса;
3. формирование плана питания и генерация списка покупок;
4. публикация отзыва, создание жалобы, получение уведомления;
5. обращение к ИИ-ассистенту через серверный прокси.

Результаты тестирования подтвердили работоспособность основных сценариев использования системы и согласованность работы клиентской и серверной частей.

3.2 Развертывание на сервере

Сборка и контейнеризация серверной части

Для унификации окружения и воспроизводимости развертывания приложение упаковано в Docker-контейнеры [29]. Проект включает:

1. контейнер базы данных PostgreSQL;
2. контейнер backend-сервиса (Django + Gunicorn);
3. контейнер frontend-сервиса (Vite + React).

Конфигурация backend-контейнера приведена в листинге 3.1.

Листинг 3.1 – Dockerfile серверной части

```
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r /app/requirements.txt
COPY . /app
EXPOSE 8000
CMD ["gunicorn", "config.wsgi:application", "--bind",
↪  "0.0.0.0:8000"]
```

Конфигурация frontend-контейнера приведена в листинге 3.2. Сборка выполняется через `npm ci`, а запуск dev-сервера — через Vite [49, 35, 50].

Листинг 3.2 – Dockerfile клиентской части

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json /app/
RUN npm ci
COPY . /app
EXPOSE 5173
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0",
↪  "--port", "5173"]
```

Оркестрация сервисов

Для запуска полного стенда используется `docker-compose.yml`, включающий базы данных, backend и frontend (листинг 3.3) [30].

Листинг 3.3 – Фрагмент docker-compose.yml

```yaml
services:
  db:
    image: postgres:16-alpine
  backend:
    build: ./Backend
    ports: ["8000:8000"]
    command: >
      sh -c "python manage.py migrate &&
             gunicorn config.wsgi:application --bind
              ↪  0.0.0.0:8000"
  frontend:
    build: ./Frontend
    ports: ["5173:5173"]
```

Порядок запуска

Последовательность развертывания:

1. перейти в корневую директорию проекта;
2. скачать (при необходимости обновить) образы командой `docker compose pull`;
3. выполнить сборку контейнеров командой `docker compose build`;
4. запустить окружение командой `docker compose up -d`;
5. проверить доступность сервисов: frontend (`http://localhost:5173`), backend API (`http://localhost:8000/api/health/`).

Такой подход сокращает время подготовки среды и обеспечивает идентичное поведение приложения на разных хостах.

3.3 Перспективы развития системы

Дальнейшее развитие FoodPlanner целесообразно выполнять по следующим направлениям:

1. расширение доменной персонализации (сезонность, бюджет, расширенные медицинские ограничения);
2. развитие рекомендательных алгоритмов и повышение качества ИИ-консультаций;
3. интеграция с внешними сервисами (трекеры активности, носимые устройства, продуктовые каталоги);
4. расширение аналитики пользовательского прогресса и поведенческих метрик;
5. усиление контуров качества: автоматизированные e2e-тесты, CI/CD и централизованная телеметрия.

3.4 Выводы по главе

В третьей главе представлены результаты тестирования, подтверждающие работоспособность ключевых пользовательских и служебных сценариев системы. Показано развертывание приложения в контейнерной среде Docker с использованием единого файла оркестрации сервисов. Также определены приоритетные направления дальнейшего развития, направленные на повышение качества персонализации, масштабируемости и эксплуатационной зрелости платформы.

ЗАКЛЮЧЕНИЕ

Выполнение выпускной квалификационной работы базировалось на анализе предметной области, сравнительном исследовании существующих решений, проектировании архитектуры системы и практической реализации программного продукта.

В ходе выполнения работы были решены следующие задачи:

1. выполнен анализ предметной области планирования питания и выявлены ключевые проблемы целевой аудитории;
2. проведен анализ аналогов и конкурентов, определены ограничения существующих подходов;
3. спроектированы бизнес-процессы и формализованы сценарии взаимодействия пользователей с системой;
4. сформированы бизнес-, функциональные и нефункциональные требования к разрабатываемой системе;
5. спроектирована архитектура веб-приложения и определена ролевая модель доступа;
6. разработана структура базы данных и описаны связи доменных сущностей;
7. разработаны макеты и реализован пользовательский интерфейс клиентской части;
8. реализованы серверная и клиентская части приложения, включая контуры персонализации, модерации, отзывов, уведомлений и ИИ-ассистента;
9. выполнено тестирование ключевых пользовательских и административных сценариев;
10. описаны и реализованы подходы к контейнеризации и развертыванию системы.

По результатам выполненной работы разработанное веб-приложение FoodPlanner демонстрирует работоспособность заявленных сценариев и мо-

жет использоваться в качестве основы для дальнейшего внедрения и развития.

В качестве перспектив развития рассматриваются расширение аналитического контура, развитие инструментов модерации и администрирования, а также усиление персонализации рекомендаций на основе пользовательского поведения.

Таким образом, поставленные задачи решены, а цель выпускной квалификационной работы — разработка веб-приложения для планирования питания с учетом предпочтений пользователей — достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. World Health Organization. World Health Statistics 2025: Monitoring Health for the SDGs / World Health Organization. – Geneva: World Health Organization, 2025. – 76 p.

2. Healthy Diet. – 2026. – [Электронный ресурс]. https://www.who.int/news-room/fact-sheets/detail/healthy-diet.

3. Здоровое питание. – 2026. – [Электронный ресурс]. https://www.takzdorovo.ru/profilaktika/zozh/dostupno-i-polezno/.

4. Рекомендации по здоровому питанию. – 2026. – [Электронный ресурс]. https://здоровое-питание.рф/.

5. 10 лучших дневников питания в телефоне. – 2026. – [Электронный ресурс]. https://ncn.nutristeppe.com/blog/10-luchshih-dnevnikov-pitaniya-v-telefone.

6. 10 лучших приложений по правильному питанию. – 2026. – [Электронный ресурс]. https://institutfitnesa.by/10-luchshih-prilozheniy-po-pravilnomu-pitaniyu/.

7. 5 лучших приложений, чтобы следить за питанием и здоровьем. – 2026. – [Электронный ресурс]. https://www.sportmaster.ru/media/articles/45888845/.

8. Главное, что нужно знать о белках, жирах и углеводах. – 2026. – [Электронный ресурс]. https://sektascience.com/food/all-about-bzhu/.

9. Кириллина, Ю. В. Моделирование бизнес-процессов / Ю. В. Кириллина, И. А. Семичастнов. – МИРЭА – Российский технологический университет, 2022. – [Электронный ресурс]. https://e.lanbook.com/book/256733.

10. Миндалёв, И. В. Моделирование бизнес-процессов с помощью IDEF0, DFD, BPMN за 7 дней / И. В. Миндалёв. – Красноярский государственный аграрный университет, 2016. – [Электронный ресурс]. https://e.lanbook.com/book/103833.

11. SADT Overview and Origins. – 2026. – [Электронный ресурс]. https://www.idef.com/idef0/sadt-history/.

12. Integration Definition for Function Modeling (IDEF0), FIPS PUB 183. – 1993. – [Электронный ресурс]. https://www.nist.gov/publications/integration-definition-function-modeling-idef0.

13. Зуева, А. Н. Моделирование бизнес-процессов в нотации BPMN 2.0: учебное пособие / А. Н. Зуева. – МИРЭА – Российский технологический университет, 2021. – [Электронный ресурс]. https://e.lanbook.com/book/176564.

14. Business Process Model and Notation (BPMN) Version 2.0.2. – 2014. – [Электронный ресурс]. https://www.omg.org/spec/BPMN/2.0.2/.

15. Поварёнок.ру. – 2026. – [Электронный ресурс]. https://www.povarenok.ru/.

16. Еда.ру. – 2026. – [Электронный ресурс]. https://eda.ru/.

17. MyFitnessPal. – 2026. – [Электронный ресурс]. https://www.myfitnesspal.com/.

18. SuperCook. – 2026. – [Электронный ресурс]. https://www.supercook.com/.

19. Овкусе.ру. – 2026. – [Электронный ресурс]. https://ovkuse.ru/.

20. OWASP Top 10:2021. – 2021. – [Электронный ресурс]. https://owasp.org/www-project-top-ten/.

21. OWASP Cross-Site Request Forgery Prevention Cheat Sheet. – 2026. – [Электронный ресурс]. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html.

22. OWASP Cross Site Scripting Prevention Cheat Sheet. – 2026. – [Электронный ресурс]. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.

23. OWASP SQL Injection Prevention Cheat Sheet. – 2026. – [Электронный ресурс]. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html.

24. django-cors-headers Documentation. – 2026. – [Электронный ресурс]. https://github.com/adamchainz/django-cors-headers.

25. Python Documentation. – 2026. – [Электронный ресурс]. https://docs.python.org/3/.

26. Django Documentation. – 2026. – [Электронный ресурс]. https://docs.djangoproject.com/en/stable/.

27. Django REST Framework Documentation. – 2026. – [Электронный ресурс]. https://www.django-rest-framework.org/.

28. PostgreSQL Documentation. – 2026. – [Электронный ресурс]. https://www.postgresql.org/docs/.

29. Docker Documentation. – 2026. – [Электронный ресурс]. https://docs.docker.com/.

30. Docker Compose Documentation. – 2026. – [Электронный ресурс]. https://docs.docker.com/compose/.

31. React Documentation. – 2026. – [Электронный ресурс]. https://react.dev/.

32. TypeScript Documentation. – 2026. – [Электронный ресурс]. https://www.typescriptlang.org/docs/.

33. Effector Documentation. – 2026. – [Электронный ресурс]. https://effector.dev/.

34. Mantine Documentation. – 2026. – [Электронный ресурс]. https://mantine.dev/.

35. Vite Documentation. – 2026. – [Электронный ресурс]. https://vite.dev/guide/.

36. Java Documentation. – 2026. – [Электронный ресурс]. https://docs.oracle.com/en/java/.

37. Node.js Documentation. – 2026. – [Электронный ресурс]. https://nodejs.org/docs/latest/api/.

38. The Go Programming Language Documentation. – 2026. – [Электронный ресурс]. https://go.dev/doc/.

39. django-environ Documentation. – 2026. – [Электронный ресурс]. https://django-environ.readthedocs.io/.

40. Psycopg 3 Documentation. – 2026. – [Электронный ресурс]. https://www.psycopg.org/psycopg3/docs/.

41. drf-spectacular Documentation. – 2026. – [Электронный ресурс]. https://drf-spectacular.readthedocs.io/.

42. Gunicorn Documentation. – 2026. – [Электронный ресурс]. https://docs.gunicorn.org/.

43. Fielding, Roy T. Architectural Styles and the Design of Network-based Software Architectures. – PhD thesis, University of California, Irvine. – 2000. – [Электронный ресурс]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

44. RFC 9110: HTTP Semantics. – 2022. – [Электронный ресурс]. https://www.rfc-editor.org/rfc/rfc9110.

45. OpenAPI Specification 3.1.0. – 2021. – [Электронный ресурс]. https://spec.openapis.org/oas/v3.1.0.

46. Insomnia Documentation. – 2026. – [Электронный ресурс]. https://docs.insomnia.rest/.

47. Postman Learning Center. – 2026. – [Электронный ресурс]. https://learning.postman.com/.

48. React Router Documentation. – 2026. – [Электронный ресурс]. https://reactrouter.com/.

49. npm CLI Documentation. – 2026. – [Электронный ресурс]. https://docs.npmjs.com/cli/v10/commands/npm-ci.

50. Vite Deployment Guide. – 2026. – [Электронный ресурс]. https://vite.dev/guide/static-deploy.html.